

班 级 2203054w

学 号 22009201185

西安电子科技大学

本科毕业设计论文



题 目 智能客服机器人的设计与交互优化

学 院 计算机科学与技术学院

专 业 计算机科学与技术

学生姓名 王奇

导师姓名 魏昱恒、Harris H M XIE

毕业设计（论文）诚信声明书

本人声明：本人所提交的毕业论文《智能客服机器人的设计与交互优化》是本人在指导教师指导下独立研究、写作的成果，论文中所引用他人的无论以何种方式发布的文字、研究成果，均在论文中加以说明；有关教师、同学和其他人员对本文的写作、修订提出过并为我在论文中加以采纳的意见、建议，均已在我的致谢辞中加以说明并深致谢意。

本论文和资料若有不实之处，本人承担一切相关责任。

论文作者：_____（签字） 时间： 年 月 日

指导教师已阅：_____（签字） 时间： 年 月 日

摘要

随着银行业务线上化和服务渠道智能化的发展，传统人工客服在高频、重复业务中面临响应压力大、服务效率不稳定等问题。本文设计并实现了面向个人银行业务的智能客服机器人原型系统 BankCopilot。系统采用 Vue 3、Element Plus、ECharts、Spring Boot、MyBatis 和 MySQL 构建前后端业务能力，并接入通义千问大模型完成自然语言意图解析与分析性文本生成。系统实现了 AI 查余额、AI 查流水、AI 账单统计与分析、AI 周报/月报、AI 智能转账以及语音输入等功能。设计上，大模型只负责理解输入和组织表达，语音识别负责将用户语音转写为文本，账户查询、流水查询、统计计算和转账执行均由后端确定性服务完成。测试结果表明，系统能够支持文本输入和语音转文字输入形式的银行客服交互，并在安全性、可解释性和可演示性方面具有一定实用价值。

关键词： 智能客服机器人 自然语言处理 银行业务 账单分析 交互优化

ABSTRACT

With the development of online banking services and intelligent service channels, manual customer service faces increasing pressure in high-frequency and repetitive scenarios. This thesis designs and implements BankCopilot, an intelligent customer service robot prototype for personal banking services. The system uses Vue 3, Element Plus, ECharts, Spring Boot, MyBatis and MySQL to build front-end and back-end business capabilities, and uses the Qwen large language model for natural language intent parsing and analytical response generation. The implemented AI functions include balance inquiry, transaction inquiry, bill statistics and analysis, weekly or monthly brief generation, intelligent transfer confirmation, and speech-to-text input. The model is responsible for understanding input and organizing natural language expressions, speech recognition converts user speech into text, while account query, transaction query, statistical calculation and transfer execution are completed by deterministic back-end services. The test results show that the system can support text input and speech-to-text banking service interaction and has practical value in security, explainability and demonstration.

Keywords: intelligent customer service robot natural language processing
banking service bill analysis interaction optimization

目录

摘 要	i
ABSTRACT.....	iii
目录	v
第一章 绪论.....	1
1.1 研究背景与意义	1
1.2 课题目标与研究内容	2
1.3 本文组织结构	3
第二章 相关研究与行业实践分析.....	5
2.1 传统银行管理系统与核心银行系统	5
2.2 银行 AI 应用的行业实践	5
2.3 银行 AI 系统面临的关键问题	6
2.4 本文系统与行业系统的差异	7
2.5 本文针对课题问题的优化措施	8
第三章 相关技术与理论基础.....	11
3.1 大语言模型与自然语言意图识别	11
3.2 自然语言处理方案选择与比较	12
3.3 Spring Boot 后端技术	13
3.4 Vue 与前端交互技术	13
3.5 数据持久化与统计分析技术	14
第四章 系统需求分析.....	17
4.1 系统总体需求	17
4.2 AI 核心功能需求	18
4.3 非功能性需求	18
第五章 系统总体设计.....	21
5.1 系统架构设计	21
5.2 AI 模块设计	22
5.3 数据库设计	23
5.4 接口与交互设计	24
第六章 AI 智能客服核心功能设计与实现	27
6.1 AI 请求处理总体流程	27

6.2 AI 语音输入功能实现	29
6.3 AI 查余额功能实现	30
6.4 AI 查流水功能实现	30
6.5 AI 账单统计与分析功能实现	32
6.6 AI 周报/月报功能实现	35
6.7 AI 智能转账功能实现	36
第七章 系统测试与交互优化分析	41
7.1 测试环境与测试方法	41
7.2 意图识别准确性测试	41
7.3 功能测试	43
7.4 交互优化设计分析	45
7.5 测试结论	47
第八章 结论与展望	49
8.1 主要结论	49
8.2 实践价值	50
8.3 不足与展望	50
第九章 结束语	53
致谢	55
参考文献	57
附录 A 主要接口与数据结构	59
A.1 AI 聊天请求结构	59
A.2 AI 聊天响应结构	59
A.3 统计查询结构	59

第一章 绪论

1.1 研究背景与意义

在数字化服务持续发展的背景下，银行业务从传统柜台办理逐步转向移动端、网页端和智能终端。用户在办理业务时，常常需要查询余额、查询交易流水、了解一段时间内的收支状况，或者发起转账等操作。这些需求频率高、重复性强，若完全依赖人工客服，不仅会增加银行服务成本，也会影响用户问题响应速度。因此，构建能够理解自然语言、连接业务系统并提供即时服务的智能客服机器人，具有较强的工程意义和应用价值。

近年来，大语言模型在自然语言理解和文本生成方面表现出显著能力，使客服机器人从传统的基于规则或检索的问答系统逐步发展为能够处理开放式表达和多轮上下文对话的智能助手^[1-3]。但在银行场景中，单纯依赖大模型直接回答并不可靠。银行业务涉及账户余额、交易流水和资金转账等敏感数据，要求系统具备准确性、可审计性和安全性。因此，本课题将大语言模型作为自然语言入口和表达增强工具，将真实业务查询、统计计算和资金操作交由后端确定性服务完成，从而在智能化体验和业务安全之间取得平衡。

BankCopilot 项目以银行智能客服为场景，实现了一个面向个人账户业务的 AI 助手。系统支持用户使用文本自然语言或语音转文字方式发起查询与操作请求，能够完成余额查询、流水查询、账单统计、周报/月报生成以及智能转账确认等功能。与普通客服问答系统相比，本系统的特点在于 AI 回复并非仅来自静态问答库，而是通过意图解析后调用真实账户、流水和统计服务，再将结果组织为用户可理解的文本、图表和报表导出链接。

从银行业务应用角度看，智能客服机器人并不只是一个问答窗口，而是业务入口、交互流程和风险控制逻辑的组合。用户在传统页面中完成一次流水查询，通常需要进入交易流水页面、选择交易类型、设置起止日期、调整分页参数，再阅读表格结果；而在智能客服交互中，用户可以直接表达“查最近 5 条支出记录”或“看最近 7 天支出趋势”。系统需要将这类口语化表达转换为后端能够执行的查询条件。该转换过程既涉及自然语言理解，也涉及业务规则约束。若只关注文本生成能力而忽略业务执行过程，智能客服就容易停留在演示层面，难以成为真正可用的银行服务入口。

另一方面，银行业务对准确性和安全性要求较高。余额、流水和转账结果均

属于用户关心的敏感信息，系统不能让模型根据语言习惯自行推断或编造金额。本文的设计思路是将大模型放在“理解用户意图”和“组织分析语言”的位置，将数据库查询、统计聚合和资金变动放在传统后端服务中执行。这样既可以利用大模型提升自然语言交互的灵活性，又可以通过确定性服务保证数据来源可追溯、业务流程可控制。该思路也是本文相对于简单聊天机器人实现的主要区别。

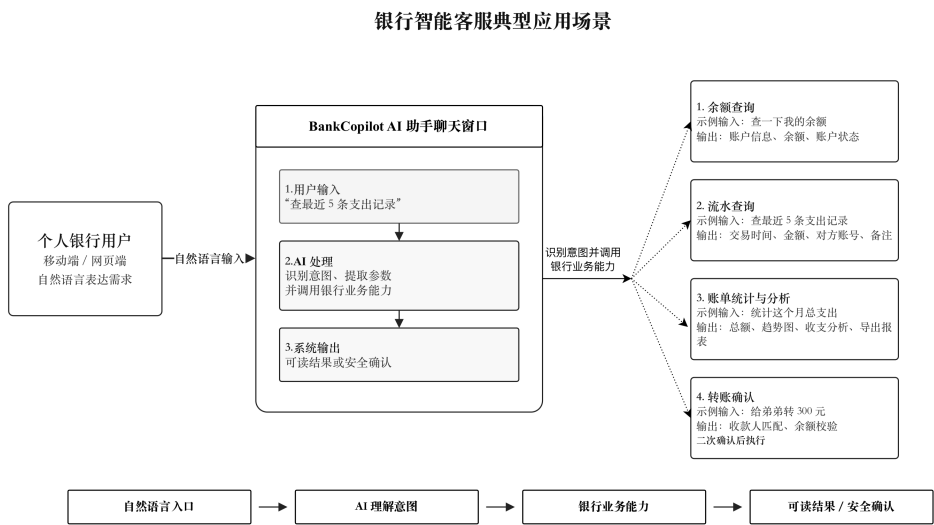


图 1.1 银行智能客服应用场景示意图

1.2 课题目标与研究内容

本课题的总体目标是设计并实现一个面向银行业务的智能客服机器人原型。课题重点研究自然语言交互、业务意图解析、对话状态管理、银行业务服务集成和前端交互优化等内容。系统以文本自然语言交互为主要方式，并补充语音转文字输入能力，围绕用户在个人银行业务中的高频需求提供智能化辅助。

结合任务书要求和项目源码实现，本文的主要研究内容包括以下几个方面。

第一，设计智能客服机器人的功能体系和交互流程，明确 AI 助手支持的业务边界。第二，选择适合项目的自然语言处理方案，通过通义千问大模型完成用户输入到结构化意图的转换。第三，设计对话管理机制，实现转账确认和流水查询补全等多轮交互。第四，将 AI 助手与后端账户、流水、统计和转账服务集成，实现真实业务数据驱动的智能回复。第五，优化前端交互展示，提供语音输入、流式响应、处理过程追踪、图表展示、快捷操作和报表导出等能力。

表 1.1 课题研究内容与系统实现对应关系

研究内容	本文实现方式
自然语言理解	通过 QwenClient 调用通义千问，将用户输入解析为 intent、时间范围、金额、收款人等结构化字段。
对话管理	通过 pendingMap 和 pendingTxnQueryMap 维护待确认转账和待补全流水查询状态。
业务系统集成	调用 AccountService、TransactionService、StatisticsService 和 PayeeService 完成真实业务处理。
交互优化	前端 AssistantPanel 实现语音输入、SSE 流式响应、AI 过程展示、图表展示、快捷确认与导出功能。
安全控制	转账前校验账户、余额和收款人信息，并要求用户二次确认。

上述研究内容共同构成了系统的 AI 能力闭环。自然语言理解负责把用户原始输入转换为可执行的业务意图，对话管理负责处理缺失信息和高风险操作确认，业务系统集成负责保证返回数据来自真实数据库，交互优化负责将处理过程和业务结果以更清晰的形式展示给用户。本文后续章节围绕这一闭环展开论述，避免把 AI 功能写成脱离系统实现的概念性描述。

课题研究范围需要明确边界。系统中的登录、注册，帐户验证等能力属于基础设施，不作为本文 AI 核心功能的重点，故不在本课题研究范围内。本文重点分析源码中已实现的语音输入、AI 查余额、AI 查流水、AI 账单统计与分析、AI 周报/月报和 AI 智能转账功能等 AI 集成特色功能。对于当前源码没有实现的消费品类识别、商户标签分析等能力，由于实现过于复杂且与 AI 集成不可控，仅在不足与展望中作为后续方向说明。

1.3 本文组织结构

本文共分为九章。第一章介绍课题背景、研究意义、研究目标和论文结构。第二章分析银行管理系统智能化发展现状、行业 AI 实践和本文系统的优化问题。第三章介绍系统开发涉及的关键技术，包括大语言模型、Spring Boot、Vue、MyBatis、MySQL、ECharts 和 SSE 流式通信等。第四章对系统需求进行分析，重点说明 AI 智能客服的核心功能需求和非功能性需求。第五章给出系统总体设计，包括系统架构、AI 模块、数据库和接口设计。第六章详细阐述语音输入、AI 查余额、AI 查流水、AI 账单统计与分析、AI 周报/月报和 AI 智能转账功能的设计与实现。第七章从功能测试和交互优化角度分析系统效果。第八章总结全文工作，并提出不足与展望。第九章为结束语。

第二章 相关研究与行业实践分析

2.1 传统银行管理系统与核心银行系统

银行管理系统首先是一个高可靠性的业务信息系统。传统银行系统通常以核心银行系统为基础，围绕账户、客户、交易、存取款、贷款和财务会计等对象组织业务流程。美国堪萨斯城联储关于核心银行系统现代化的研究指出，核心银行系统是银行处理日常交易并更新账户和财务记录的后端信息系统，其基础服务包括账户管理、客户管理、存取款处理、贷款处理和财务会计等内容，扩展服务还可能包括支付处理、客户支持、欺诈与风险管理、合规审计和报表分析等^[4]。由此可以看出，传统银行管理系统的核心目标是保证业务流程规范、交易记录准确、账户状态一致和后台服务稳定。

从软件工程角度看，传统银行管理系统更关注确定性。用户提交查询或交易请求后，系统需要根据登录身份、账户状态、交易规则和数据库记录返回明确结果。余额是多少、流水是否存在、转账是否成功，这些结果都应来自业务数据库和后端服务，而不能来自语言模型推测。本文所实现的 **BankCopilot** 虽然是本科毕业设计中的轻量级原型，但仍遵循这一基本原则：账户余额、交易流水、统计聚合和转账执行均由后端业务服务完成，AI 模块不直接产生业务事实。

但是，传统系统在人机交互方面也存在明显不足。菜单、表单和表格适合稳定业务管理，却不一定适合用户快速表达复杂查询需求。例如用户想知道“这个月支出有没有异常”时，传统页面往往需要用户手动选择时间范围、交易类型和统计方式，再自行理解表格或图表结果。若把智能客服接入传统银行管理系统，系统就可以把自然语言转化为后端查询参数，并把聚合结果组织成更容易理解的说明。这正是本文选择银行智能客服作为课题场景的原因。

2.2 银行 AI 应用的行业实践

随着金融科技和大语言模型的发展，银行业正在从传统信息管理系统向智能化服务系统演进。国际清算银行金融稳定研究所指出，人工智能正在影响金融机构的运营效率、风险管理和客户体验，同时也带来模型风险、数据隐私、生成式 AI 幻觉、治理机制和第三方服务商管理等问题^[5]。因此，银行 AI 系统不能只关注“能否回答问题”，还要关注回答依据是否可靠、业务边界是否清楚、敏感数据是否受到保护、关键操作是否能够追溯。

在客户服务方面，美国银行推出的虚拟金融助手 Erica 是银行 AI 应用的典型案例。公开资料显示，Erica 自 2018 年推出以来已服务近 5000 万用户，累计完成超过 30 亿次客户交互，并能够向用户提供账户提醒、消费洞察和个性化财务建议等服务^[6]。这一案例说明，银行 AI 的一个重要方向是把用户从复杂页面路径中解放出来，通过自然语言、主动提醒和个性化信息组织提高客户服务效率。

在业务流程自动化方面，JPMorgan Chase 的 COiN 合同智能平台体现了 AI 在信贷和合同处理场景中的价值。摩根大通年报披露，该平台能够从 12000 份年度商业信贷协议中提取 150 个相关属性，原本人工审查最多需要 36 万小时，而系统可以在数秒内完成^[7]。这类系统的价值不只是节省人工时间，更重要的是减少重复阅读和人工处理带来的遗漏，使复杂业务流程具备更高的自动化程度。

在国内银行实践中，招商银行也在多个业务场景中应用大模型和智能化工具。其公开材料显示，招商银行已将大模型应用于零售服务、普惠金融授信、交易机器人、运营管理、风险管理和反洗钱等场景，相关应用既服务客户，也服务内部员工^[8]。这些案例表明，银行 AI 的应用范围已经从单一客服问答扩展到信贷审核、风险管理、运营支持和内部知识服务等综合场景。

上述行业实践对本文具有两点启发。第一，银行 AI 的价值并不等同于聊天能力本身，而在于能否与真实业务系统结合，缩短查询路径、辅助业务判断并提高服务效率。第二，银行 AI 不能脱离治理机制，必须在隐私保护、人工复核、解释说明和操作审计方面设置边界。本文系统虽然规模远小于真实商业银行系统，但在设计思路借鉴了这些方向，重点研究如何在轻量级 Web 银行业务原型中完成 AI 与传统业务服务的结合。

2.3 银行 AI 系统面临的关键问题

银行场景中的智能化改造具有明显的双重性。一方面，AI 能够提升服务效率和交互体验；另一方面，银行业务涉及资金、身份和信用信息，错误回答和错误操作可能造成严重后果。国际清算银行关于 AI 可解释性的研究指出，可解释性关系到透明度、问责、监管合规和消费者信任^[9]。美国国家标准与技术研究院提出的 AI 风险管理框架也强调，可信 AI 应具备有效可靠、安全稳健、透明可问责、可解释、隐私增强和公平等特征^[10]。

在信贷和风险管理场景中，AI 的问题更加突出。世界银行信用评分指南指出，信用评分方法已经从传统统计技术发展 to 机器学习、随机森林、梯度提升和深度神经网络等方法，这些方法能够带来自动化、准确性和客户体验方面的改进，但也可能引发隐私、公平性、可解释性和历史偏差等问题^[11]。相关研究也表明，机

器学习在银行风险管理中具有应用价值，但模型效果、数据质量和解释能力会直接影响其落地效果^[12]。

结合本文课题，银行 AI 系统至少需要面对以下问题。第一，业务事实不能由模型编造，余额、流水和转账结果必须来自确定性业务系统。第二，高风险操作不能由模型直接完成，转账等资金操作必须经过参数校验和用户确认。第三，AI 回复需要解释依据，尤其是统计总结、异常提醒和趋势分析，应明确时间范围、指标口径和数据来源。第四，系统需要控制能力边界，不能回答源码中没有实现的功能，也不能虚构消费品类、商户标签或授信结论。第五，智能交互需要和原有页面共存，避免为了引入 AI 而破坏传统业务系统的可维护性。

因此，本文研究的问题可以概括为：如何在保证数据准确、业务可控和风险操作可确认的前提下，将自然语言交互、语音输入、统计解释和主动简报能力引入银行业务管理原型，使系统从“菜单表单式操作”扩展为“自然语言辅助操作”。这一问题既不是单纯训练一个模型，也不是简单增加一个聊天窗口，而是研究 AI 模块与传统后端业务服务之间的职责划分和交互边界。

2.4 本文系统与行业系统的差异

本文系统不能与真实商业银行的大规模 AI 平台直接等同。Bank of America、JPMorgan Chase 和招商银行等机构的 AI 系统通常建立在完整的核心银行系统、数据中台、风控平台、合规审计平台和长期运营数据之上，具有大规模用户、复杂组织流程和严格监管要求。本文系统定位为本科毕业设计中的轻量级 Web 银行智能客服原型，目标不是复现大型银行 AI 平台，而是在可控范围内验证 AI 与传统业务管理系统结合的可行路径。

表 2.1 行业银行 AI 系统与本文系统对比

对比维度	行业系统特点	本文系统定位	本文优化重点
业务范围	覆盖账户、支付、信贷、风控、合规、运营等完整链路	面向个人账户业务的轻量级银行智能客服原型	聚焦余额、流水、统计、简报和转账确认等高频场景
客户交互	通过虚拟助手、移动银行和主动提醒提供个性化服务	在 Web 前端中提供文本与语音输入入口	将菜单式操作扩展为自然语言辅助操作
数据处理	依托真实生产数据、数据仓库和风控模型	使用项目数据库中的账户、收款人和交易流水数据	坚持由后端服务提供业务事实，AI 负责理解和表达
风险控制	包含反欺诈、反洗钱、信贷评分和合规审计等体系	主要完成转账校验、二次确认和能力边界控制	将高风险操作设计为“AI 解析 + 后端校验 + 用户确认”流程
AI 治理	强调模型评估、审计、隐私保护和人工复核	在原型系统中实现过程展示、统计口径说明和安全兜底	使 AI 输出可解释、可检查，并避免超出已实现能力

从表 2.1 可以看出，本文系统的不同点主要体现在“小而精”的系统定位。系统没有追求覆盖所有银行业务，而是选择几类互不重复、能够代表不同 AI 集成方式的功能：查余额体现确定性数据查询，查流水体现条件抽取和多轮补全，账单统计体现聚合数据解释，周报/月报体现主动式信息组织，智能转账体现风险操作的分步确认，语音输入体现交互入口扩展。这些功能共同覆盖了查询、统计、分析、简报和操作确认等典型模式，能够支撑对“AI 如何嵌入传统银行管理系统”的讨论。

因此，本文的价值不在于声称系统达到商业银行平台水平，而在于借鉴行业 AI 应用思路，在本科毕业设计场景下形成一个边界清楚、实现完整、便于演示和验证的原型。通过该原型，可以观察大语言模型在银行业务系统中更适合承担哪些职责，也可以明确哪些职责仍必须由传统后端服务和用户确认机制承担。

2.5 本文针对课题问题的优化措施

针对传统银行管理系统交互路径较长、结果解释不足和智能化程度有限的问题，本文提出以下优化措施。

第一，增加智能交互入口。传统 Web 银行业务系统主要依赖菜单、表单和表

格完成操作，用户需要主动寻找功能入口。本文在系统中提供 AI 助手面板，支持文本输入和语音转文字输入。用户可以直接输入“查询最近一周流水”“这个月支出了多少”“生成本周简报”等自然语言请求，系统通过大语言模型解析意图，并由后端服务执行对应业务查询。语音输入不单独改变业务逻辑，而是把用户语音转写为文本后复用原有 AI 请求链路，从而降低用户输入成本。

第二，明确大模型与后端服务的职责边界。本文没有让大模型直接回答余额、流水和转账结果，而是把大模型限定为意图解析器和表达组织器。账户数据、流水数据、统计数据 and 资金变动均由 Spring Boot 后端服务从数据库中查询或执行。这样可以避免模型幻觉直接影响业务事实，也使每一次查询和操作都能够回到明确的服务方法和数据表。

第三，优化统计解释与主动简报能力。传统流水页面通常只提供明细表格，用户需要自行理解收支情况。本文在账单统计与周报/月报功能中增加时间范围、指标口径、趋势数据、异常提醒和快捷追问等展示内容，使系统能够从“返回数据”进一步发展到“解释数据”。对于用户而言，这种设计减少了阅读表格和手动计算的 cost；对于论文研究而言，它体现了 AI 在信息组织和表达优化方面的作用。

第四，优化风险操作流程。转账是银行业务中安全要求较高的操作。本文将智能转账设计为分步流程：首先由 AI 解析收款人、金额和备注等参数；然后后端校验账户、余额、收款人和历史转账信息；最后由前端展示确认按钮，用户确认后才真正执行转账。该流程避免了 AI 一次性直接完成资金操作，也保留了传统业务系统中必要的确认环节。

第五，增加可解释和过程展示机制。本文在 AI 回复中保留 `trace`、`steps`、`explain`、`confidence`、`tool` 和 `status` 等过程字段，前端可展示意图、处理步骤和统计口径。这样用户不只看到最终回答，也能看到系统为何这样回答、调用了哪类能力、统计范围是什么。对于银行智能客服而言，过程可见比单纯回复自然更重要，因为它能帮助用户判断结果是否可信。

第六，建立能力边界和安全兜底。系统提示词中明确限制模型不得输出未实现能力，对于消费品类、商户标签、复杂风控结论等源码中不存在的数据，系统不把它写成已实现成果。同时，后端对模型输出进行结构化校验，并在模型服务不可用或输出异常时采用规则化兜底回复。这种设计使系统在演示和测试中更稳定，也符合银行业务系统对可控性的要求。

第七，设置可验证的优化评价方式。本文不使用行业案例中的效率数据来描述本系统效果，而是通过功能测试和交互分析验证原型是否达到设计目标。评价重点包括：自然语言请求能否被正确解析，余额和流水结果是否与数据库一致，

统计口径是否清楚，转账是否经过二次确认，语音输入是否能进入原有文本链路，AI 回复是否避免未实现能力。通过这些指标，可以更客观地说明本文优化措施的有效性。

综合来看，本文的创新点主要体现在三个方面。第一，在传统银行业务管理原型之上引入文本与语音自然语言入口，使系统从表单式操作扩展为自然语言辅助操作。第二，将大语言模型与确定性后端服务分工，使 AI 负责理解和表达，业务服务负责事实和执行。第三，围绕银行业务安全要求设计过程展示、统计解释、二次确认和安全兜底机制，使智能客服不仅能回答问题，也能以可解释、可控制的方式辅助完成业务。

第三章 相关技术与理论基础

3.1 大语言模型与自然语言意图识别

自然语言意图识别是智能客服系统的入口能力，其目标是从用户的自然语言表达中识别用户想要完成的任务，并抽取执行任务所需的关键槽位信息。传统方法通常采用规则匹配、关键词识别或监督学习分类模型。随着大语言模型的发展，模型能够更好地理解口语化表达和复杂句式，适合用于将用户输入转换为结构化的意图表示。

本系统通过 QwenClient 封装通义千问兼容 OpenAI Chat Completions 的调用方式。在 AssistantService 中，parseIntentByQwen 方法将用户输入和系统提示词发送给模型，要求模型返回严格 JSON。该 JSON 包含 intent、toAccountNo、payeeAlias、amount、description、txnType、timeRange、metric、groupBy 等字段。后端随后根据 intent 分发到不同业务处理方法。

需要强调的是，在银行业务场景下，大语言模型无法直接作为业务事实来源。本系统将大模型限定为意图解析器和分析文本生成器，余额、流水、统计值和转账执行均来自后端业务服务。这种设计能够保留自然语言交互的灵活性，同时降低模型幻觉带来的业务风险。

在银行智能客服场景中，意图识别的目标不是单纯判断用户属于哪一个聊天主题，而是把自然语言输入转换为后端服务能够理解的参数组合。例如“这个月花了多少钱”需要被转换为统计意图、支出方向、按月时间范围和求和指标；“给弟弟转 300 元”需要被转换为转账意图、收款人昵称和金额。意图解析结果越接近后端接口参数，业务链路越短，系统越容易验证和维护。因此本文将模型输出限定为结构化 JSON，并在服务层进行二次校验。

大语言模型还承担分析性表达生成任务。对于统计结果，后端已经计算出总额、趋势和对比数据，直接返回数字给用户并不利于理解，因此，系统会把统计事实整理后交给模型生成自然语言总结，使回复更符合客服场景的表达习惯。需要强调的是，模型生成的分析只能基于后端提供的事实数据，不能扩展出数据库中不存在的交易类型、商户类别或风险结论。

3.2 自然语言处理方案选择与比较

任务书要求本课题研究并选择适用的自然语言处理和机器学习算法。结合银行智能客服场景，本文在系统设计阶段比较了关键词规则匹配、传统文本分类模型、检索式 FAQ 问答、大语言模型意图解析和情感分析等方案。由于本系统的核心目标不是开放式闲聊，而是把用户自然语言请求转换为后端可执行的银行业务意图，因此方案选择重点关注意图识别能力、槽位抽取能力、扩展成本和业务可控性。

表 3.1 自然语言处理方案比较

方案	优点	不足	本文采用情况
关键词规则匹配	实现简单、结果可控、便于定位问题	对口语化表达、同义表达和复杂句式适应性较弱，规则数量增加后维护成本较高	用于不支持能力拦截和部分兜底判断
传统文本分类模型	可通过训练集计算准确率、召回率等指标，适合固定意图分类	需要标注语料，新增意图或槽位时需要重新训练或补充样本	未作为主方案
检索式 FAQ 问答	适合常见问题和制度说明，答案稳定	难以直接完成余额、流水、统计和转账等实时业务查询	可作为知识库扩展方向
大语言模型意图解析	对口语表达、复杂句式和多槽位抽取适应性较强，可直接输出结构化 JSON	存在格式不稳定、幻觉和超出能力边界表达的风险	作为本文主方案
情感分析	可判断用户情绪，用于人工客服优先级或投诉识别	当前系统重点是账户查询、统计分析和转账确认，情绪判断不会直接影响核心业务执行	暂不实现，仅作为后续客服扩展方向

从表 3.1 可以看出，传统文本分类模型虽然更便于进行标准化机器学习评估，但它要求预先准备较多标注语料。本课题是本科毕业设计原型，系统意图数量较少，但每类意图都包含多个槽位。例如查流水不仅需要识别 `transactions` 意图，还需要抽取收入或支出、时间范围、查询条数和是否需要补全月份；智能转账不仅需要识别 `transfer` 意图，还需要抽取收款人、金额、备注和历史金额复用方式。若采用普通文本分类模型，还需要额外设计槽位抽取规则，整体开发成本较高。

检索式 FAQ 问答适合回答“如何开卡”“密码忘记怎么办”等常见问题，但本文系统的重点是连接真实业务数据。余额、流水和统计结果必须来自数据库，转账必须经过后端校验和用户确认，单纯检索固定答案无法满足这些场景。因此，

本文没有把 FAQ 检索作为核心方案，而是将其视为后续知识库扩展方向。

综合比较后，本文采用“大语言模型意图解析 + 后端确定性服务 + 规则兜底”的混合方案。AssistantService 中的 parseIntentByQwen 方法要求通义千问输出严格 JSON，将用户输入解析为 intent、txnType、timeRange、amount、payeeAlias、metric 和 groupBy 等字段；后端再根据 intent 分发到余额、流水、统计和转账处理流程。对于消费分类、商户类型、场景标签等当前系统不支持的能力，后端通过规则拦截强制输出 other，避免模型把未实现能力说成已实现能力。

本文暂未实现情感分析，原因在于当前课题目标聚焦银行业务办理效率和交互流程优化，而不是客服坐席调度或投诉情绪识别。余额查询、流水查询、统计分析和转账确认等核心功能主要依赖意图识别、槽位抽取和业务服务调用，情绪极性不会改变后端业务执行路径。若后续系统扩展到人工客服分流、投诉处理或服务满意度分析，再引入情感分析会更合理。

3.3 Spring Boot 后端技术

Spring Boot 是系统后端的主体框架，用于快速构建 RESTful Web 服务^[13]。项目后端以 Controller、Service、Mapper 分层组织代码。其中 AssistantController 提供 AI 相关接口，AssistantService 完成 AI 请求编排，AccountService、TransactionService、StatisticsService 和 PayeeService 提供账户、流水、统计和收款人相关业务能力。

后端同时使用 Spring MVC 拦截器实现 Token 校验，通过 TokenInterceptor 解析 JWT 并将当前用户 id 保存到 CurrentHolder。AI 接口在调用业务服务前会读取当前用户 id，确保查询和操作均在登录用户上下文中执行。虽然登录鉴权不是本文重点，但它是 AI 查询个人账户数据和执行转账操作的必要基础。

从系统稳定性角度看，Spring Boot 后端承担了业务可信边界的角色。大模型服务可能因为网络、接口限流或返回格式异常而产生不确定性，但后端服务必须保证接口输入、权限边界、金额计算和数据库更新的确定性。AssistantService 在调用模型之后，会根据解析结果进入固定分支，再调用相应业务服务。这样的设计使模型能力被控制在服务编排层，而不会绕过后端已有的校验逻辑。

3.4 Vue 与前端交互技术

前端采用 Vue 3 构建组件化界面，使用 Element Plus 提供表单、按钮、抽屉、弹窗、表格等基础组件^[14]。AI 助手界面主要由 AssistantPanel.vue 实现，该组件同时用于首页主面板和右上角智能客服抽屉。

在 AI 交互方面，前端通过 `fetch` 调用 `/assistant/chat/stream` 接口，读取服务端发送事件流，并根据 `trace`、`message`、`error`、`done` 等事件更新界面。对于统计分析结果，前端根据 `chartType` 和 `chartData` 使用 `ECharts` 渲染柱状图、折线图、饼图和概览图^[15]。对于转账确认，前端根据 AI 回复内容显示“确认”和“取消”快捷按钮，从而降低用户输入成本。

在输入方式方面，`AssistantPanel` 还使用浏览器提供的 `SpeechRecognition` 或 `webkitSpeechRecognition` 能力实现中文语音识别。该功能不单独训练语音模型，而是利用浏览器语音识别接口将用户语音转写为文本，再填入聊天输入框。后续处理仍然沿用原有 AI 文本请求链路，因此语音输入本质上是对自然语言输入入口的扩展。

前端交互设计直接影响智能客服的可用性与交互性。若系统只在页面底部显示一段长文本，用户很难判断 AI 是否真正调用了业务服务，也难以理解统计结果的口径。因此 `AssistantPanel` 在展示回复的同时，还展示处理过程、图表、快捷按钮和导出入口。对于答辩展示场景，这些可视化元素也能更直观地呈现系统的 AI 能力，而不是停留在控制台或接口返回层面。

3.5 数据持久化与统计分析技术

系统使用 `MySQL` 存储用户、账户、收款人和交易流水数据。`MyBatis` 用于完成 SQL 映射^[16]，`TransactionMapper.xml` 中定义了流水分页查询、总金额统计、笔数统计、最大交易查询、趋势聚合、收入支出对比、按对方账号聚合以及历史转账流水匹配等 SQL。

统计分析功能由 `StatisticsServiceImpl` 负责，根据前端或 AI 解析出的 `StatisticsQueryDTO` 选择 `sum`、`count`、`max`、`trend` 等统计方式。对于支出金额，SQL 中使用 `ABS(amount)` 将支出负数转为正向统计值，便于用户理解。统计结果再由 `AssistantService` 组织为事实文本，交给大语言模型生成分析总结，或者在模型不可用时使用规则化兜底总结。

数据持久化层为多项 AI 功能提供事实依据。余额查询依赖账户表，流水查询和统计分析依赖交易流水表，智能转账依赖常用收款人表和账户表，历史金额复用依赖历史交易记录。不同功能虽然在前端都表现为聊天输入，但在后端对应的数据读取和更新路径并不相同。论文在设计与实现章节中将分别说明这些路径，体现该系统不是简单拼接模型接口，而是围绕银行业务数据建立完整功能链路。

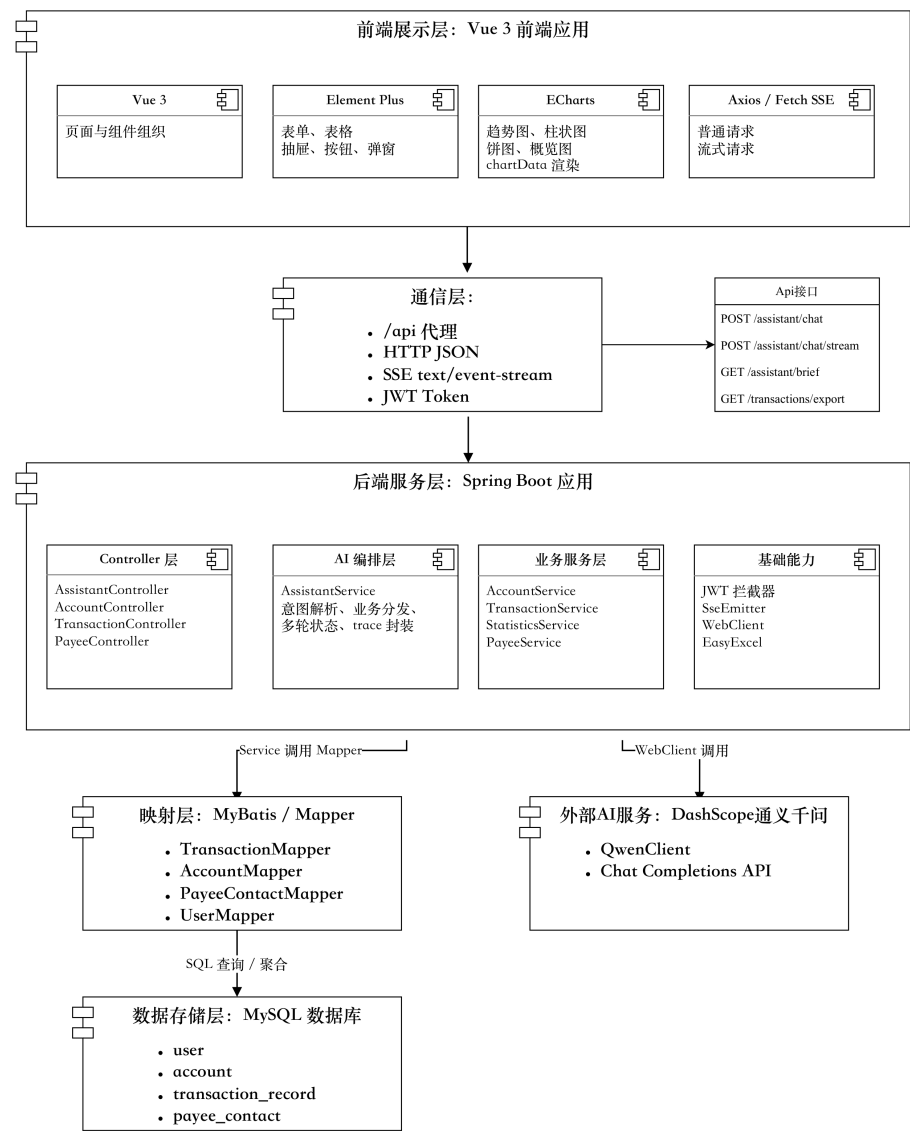


图 3.1 系统技术栈结构图

第四章 系统需求分析

4.1 系统总体需求

BankCopilot 面向银行个人用户，目标是通过智能客服机器人降低用户完成常见银行业务的操作复杂度。用户不需要记忆具体菜单和筛选条件，而是可以使用自然语言表达需求。系统应能够识别用户意图，将其映射到后端业务服务，并以文本、图表或导出文件等形式返回结果。

从用户角度看，系统需要支持账户余额查询、交易流水查询、收支统计分析、周报/月报查看和转账确认等能力。从系统角度看，AI 助手需要具备意图识别、槽位抽取、时间解析、多轮状态管理、业务服务调用、结果格式化和前端可视化展示等能力。

总体需求还体现在系统边界的划分上。AI 助手需要为用户提供更自然的入口，但不能替代后端业务规则。用户输入越口语化，系统越需要在后端建立明确的转换规则和校验逻辑。例如，用户可以说“这个月花了多少”，但后端必须把它转换为确定的起止时间、交易方向和统计指标；用户可以说“给弟弟转 300”，但后端必须识别收款人、校验金额、检查余额并等待确认。只有把自然语言表达和确定性业务规则结合起来，系统才能在银行场景中具备可用性。

此外，系统需满足本科毕业设计展示和答辩评阅的需要，在查看系统时，不仅需要看到 AI 回复文本，还需要看到它调用了什么业务能力、依据了哪些数据、是否存在安全确认环节。因此，系统在需求层面就需要考虑处理过程展示、统计图表展示和快捷操作按钮。它们不是额外装饰，而是证明系统具有真实业务链路的重要交互证据。

4.2 AI 核心功能需求

表 4.1 AI 核心功能需求表

功能编号	功能名称	需求描述
F1	AI 查余额	用户通过自然语言查询账户余额和账户基本信息，系统返回户名、账号、账户类型、状态和余额。
F2	AI 查流水	用户通过自然语言查询交易记录，系统支持收入/支出筛选、时间范围和查询条数解析。
F3	AI 账单统计与分析	用户查询总金额、笔数、最大交易或趋势时，系统基于数据库统计结果生成文本分析和图表数据。
F4	AI 周报/月报	用户打开 AI 面板时，系统展示本周或本月收入、支出、净流入、趋势、对比和异常提醒。
F5	AI 智能转账	用户通过自然语言发起转账，系统解析收款人、金额和备注，校验后等待用户确认再执行。
F6	AI 语音输入	用户点击语音输入按钮后，系统识别中文语音并回填到聊天输入框，后续按普通文本请求继续处理。

除上述功能外，系统还需要提供 AI 基础设施能力，包括自然语言意图识别、时间范围纠偏、SSE 流式响应、AI 处理过程展示、统计口径解释和 Excel 导出链接生成等。这些能力本身不是用户直接提出的业务目标，但对提升交互体验和系统可解释性具有重要作用。

4.3 非功能性需求

安全性方面，系统必须保证用户只能查询自己的账户和流水数据，资金流动操作必须经过校验和二次确认。AI 转账不能在用户首次输入后直接扣款，必须先生成确认信息，待用户回复“确认”后才能执行。对方账号在聊天回复中应进行脱敏展示。

准确性方面，系统中的账户余额、流水记录和统计金额必须来自数据库查询结果，不得由大模型直接编造。模型输出的统计总结应受到后端事实约束，若模型返回不合适内容，系统需要使用规则化总结兜底。

可用性方面，系统需要支持口语化输入，如“查一下我的余额”“看最近 7 天支出趋势”“给弟弟转 300 元”等。对于缺失关键信息的请求，系统应通过多轮对话引导用户补全，而不是直接报错。

交互性方面，AI 助手应尽量减少用户等待感和理解成本。系统通过流式响应

展示处理过程，通过图表展示统计结果，通过快捷按钮完成转账确认或简报追问，从而提高交互效率。

可维护性方面，系统应保持前后端职责清晰。前端负责交互展示和用户输入收集，后端负责业务判断和数据访问，大模型调用被封装在独立客户端中。这样做的好处是，当后续需要更换模型、调整提示词或增加统计指标时，可以优先修改 QwenClient、AssistantService 或 StatisticsService，而不需要大范围改动前端页面和数据库结构，清晰的模块划分有利于该系统设计的后续维护与迭代升级。

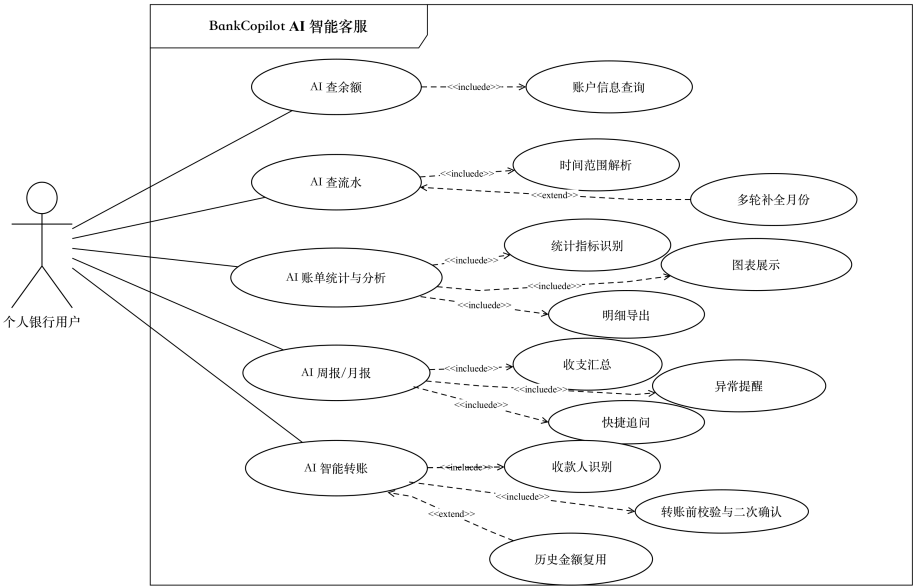


图 4.1 AI 智能客服用例图

第五章 系统总体设计

5.1 系统架构设计

系统采用前后端分离架构。前端由 **Vue 3** 应用负责页面展示和用户交互，后端由 **Spring Boot** 应用负责业务处理和数据访问，**MySQL** 负责数据持久化，**DashScope** 通义千问负责自然语言意图解析和分析性回复生成。前端通过 `/api` 代理访问后端接口，后端通过 **WebClient** 访问大模型服务。系统总体架构如图5.1 所示。

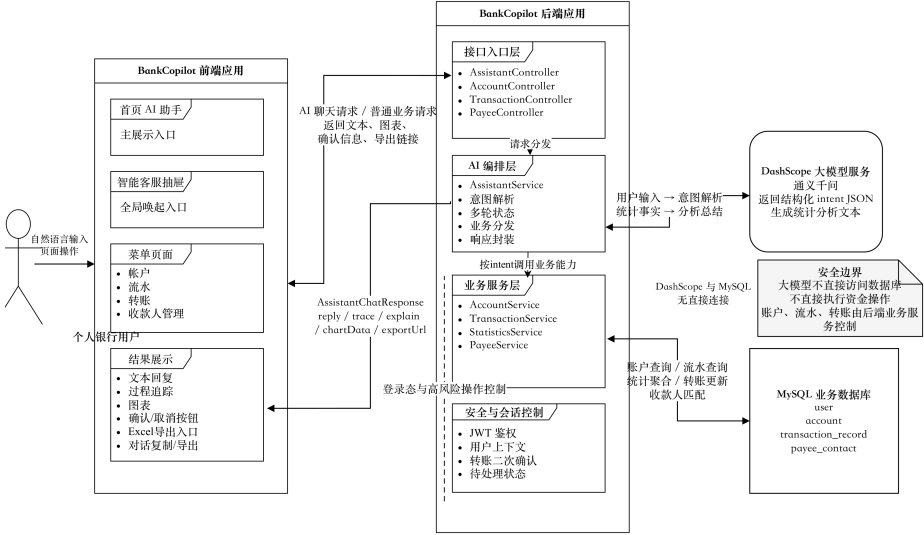


图 5.1 系统总体架构图

在该架构中，**AI 助手**并不是独立于业务系统之外的聊天模块，而是业务系统的一层自然语言交互入口。用户先通过自然语言输入页面操作，在前端应用的智能助手接入到后端应用的接口入口层，进行请求分发后进入 **AI 编排层**，通过 **DashScope** 进行意图分析出意图 **intent**，通过 **intent** 分发业务，再调用帐户，流水，统计或收款人服务，查询 **MySQL** 数据库，将结果封装后返回前端。需要强调的是，大模型无法直接访问数据库，不执行资金操作，帐户流水，转账有后端业务控制。

5.2 AI 模块设计

AI 模块由接口层、编排层、模型调用层、业务服务层和前端展示层组成。接口层对应 AssistantController，提供 /assistant/chat、/assistant/chat/stream 和 /assistant/brief 三类接口。编排层对应 AssistantService，负责处理待确认状态、调用 parseIntentByQwen、分发不同 intent、封装 trace 和 explain。模型调用层由 QwenClient 实现，统一封装通义千问请求。业务服务层包括 AccountService、TransactionService、StatisticsService 和 PayeeService。前端展示层由 AssistantPanel 负责，同时包含语音输入按钮和语音识别状态展示。

这种模块划分的核心思想是“模型不直接接触数据库，业务不依赖模型自由发挥”。如图5.2，AssistantService 处于 AI 模块的中心位置，它既要理解模型返回的意图字段，又要根据业务规则决定是否继续执行。对于普通查询，服务层可以直接调用账户或流水服务；对于缺少月份的流水查询，服务层会保存待补全状态并追问用户；对于转账请求，服务层会生成待确认对象并等待用户下一轮确认。不同业务路径虽然都从聊天输入开始，但后端处理策略并不相同。

AI 模块还承担结果适配工作。业务服务返回的通常是对象、列表或聚合数据，而用户更希望看到简明文本、图表和下一步操作入口。因此 AssistantService 会把业务结果转换为 AssistantChatResponse，前端再根据 chartType、chartData、trace、explain 和 exportUrl 等字段决定如何展示。该设计使回复内容和展示形式解耦，可以在不破坏原有接口的基础上扩展，增加新的图表类型或统计指标。

表 5.1 AI 模块主要职责

模块	源码位置	主要职责
AI 接口入口	AssistantController.java	接收聊天请求、流式聊天请求和简报请求。
AI 编排服务	AssistantService.java	意图解析、业务分发、多轮状态、统计总结。
模型调用	QwenClient.java	调用通义千问 Chat Completions 接口。
时间处理	Time.java	自然语言时间纠偏和时间范围填充。
前端面板	AssistantPanel.vue	聊天、语音输入、简报、图表、过程追踪和确认按钮展示。

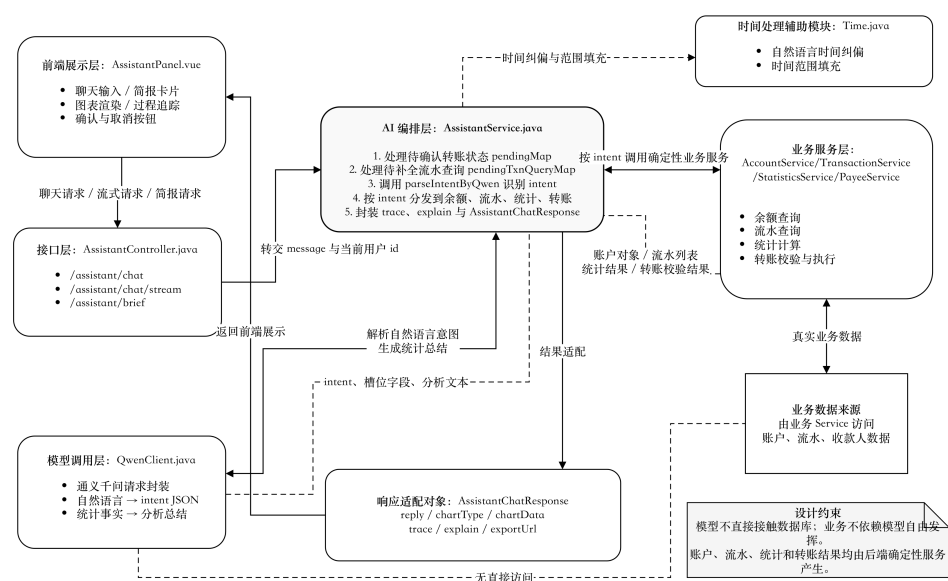


图 5.2 AI 请求处理架构图

5.3 数据库设计

系统 AI 功能依赖的核心数据表包括 account、transaction_record、payee_contact 和 user。AI 查余额主要读取 account 表和 user 表；AI 查流水和账单统计主要读取 transaction_record 表；AI 智能转账需要读取 account 表、payee_contact 表，并在执行后写入 transaction_record 表。

表 5.2 AI 功能相关数据表说明

数据表	用途
user	保存用户基本信息，为账户户名和登录用户上下文提供支持。
account	保存银行账号、账户类型、余额和状态，是查余额和转账的基础表。
transaction_record	保存收入、支出、金额、对方账号、备注和交易时间，是流水查询和统计分析的数据来源。
payee_contact	保存用户收款人、银行卡号和昵称，用于 AI 按昵称转账。

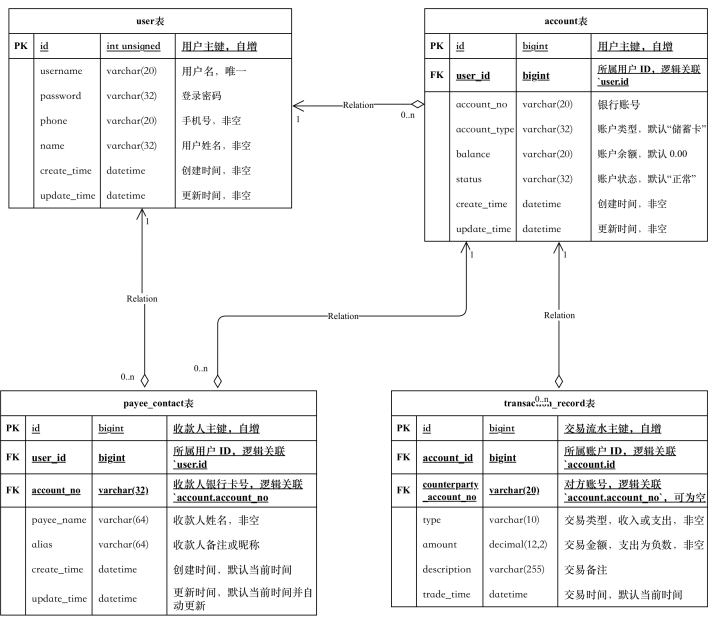


图 5.3 数据库 ER 图

5.4 接口与交互设计

AI 相关接口主要包括普通聊天接口、流式聊天接口，简报接口以及 Excel 导出接口。普通聊天接口用于非流式请求或流式失败后的降级处理；流式聊天接口用于提升用户等待体验；简报接口用于获取周报或月报数据；导出接口用于导出明细数据 Excel。

表 5.3 AI 相关接口设计表

接口	方法	功能
/assistant/chat	POST	普通 AI 聊天请求，返回完整 AssistantChatResponse。
/assistant/chat/stream	POST	SSE 流式 AI 聊天请求，返回 trace、message、done 等事件。
/assistant/brief	GET	获取 AI 周报或月报简报。
/transactions/export	GET	根据统计口径导出交易流水 Excel。

前端交互设计上，AI 面板既可作为首页主体展示，也可通过右上角智能客服抽屉打开。面板顶部展示周报/月报卡片，中间展示聊天记录和图表，底部提供文本输入框、演示脚本按钮、语音输入按钮和发送按钮。当 AI 回复等待转账确认时，前端显示确认和取消快捷按钮。

接口设计遵循前后端分离项目的基本原则。普通聊天接口适合一次性返回结果，便于调试和兜底；流式聊天接口适合实际交互，可以让前端先显示处理步骤，再显示最终回复；简报接口面向页面初始化场景，用户打开 AI 面板即可看到近期账户概览。语音识别不新增后端接口，识别结果直接进入原有聊天输入框。不同接口服务于不同交互时机，避免把所有逻辑都压在单一接口中，也便于后续测试和维护。

第六章 AI 智能客服核心功能设计与实现

6.1 AI 请求处理总体流程

AssistantService 的 chat 方法是 AI 请求处理的核心入口。该方法首先校验用户输入是否为空，然后优先处理待确认转账和待补全流水查询。如果当前用户存在未完成的多轮状态，则根据用户输入继续处理该状态；若不存在，则调用 parseIntentByQwen 解析新请求的意图。解析完成后，系统根据 intent 分发到 handleTransfer、handleTransactions、handleBalance、handleStatistics 或 fallbackChat。

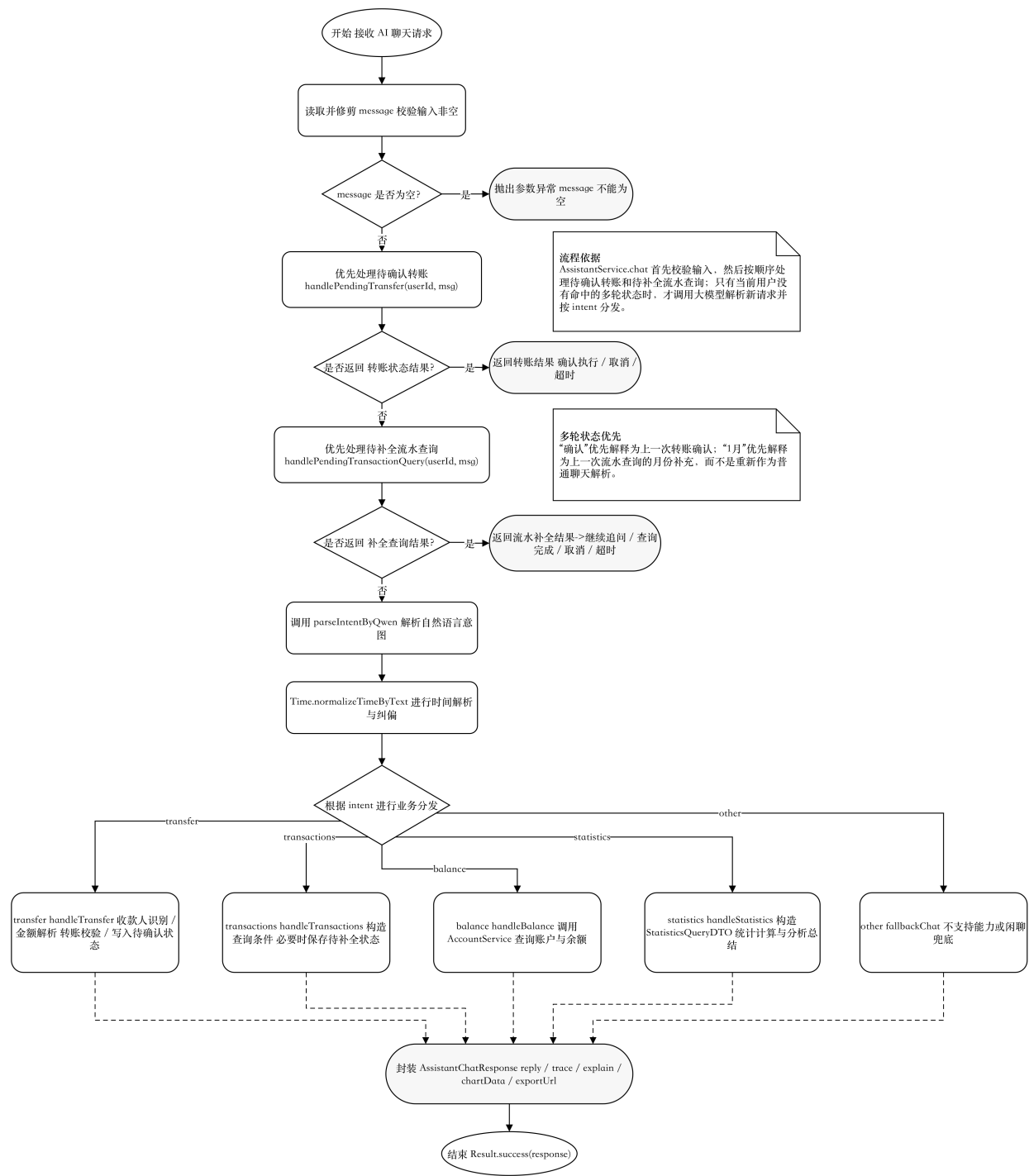


图 6.1 AI 聊天请求总体流程图

该流程体现了“多轮状态优先，新请求解析其次”的设计思路。例如用户发起转账后，系统等待“确认”或“取消”，此时下一条用户输入应优先作为转账确认处理，而不是重新解析为普通聊天请求。类似地，当用户查询“17 号收入记录”但缺少月份时，系统会等待用户补充月份，下一条“1 月”应被视为补全条件。

这种处理顺序符合聊天式业务系统的实现规律。单轮问答只需要根据当前输入返回结果，而银行客服机器人需要处理用户前后两轮输入之间的关系。如果不维护状态，系统就无法判断“确认”到底确认什么，也无法判断“1月”是在补充哪个查询。

在响应结构上，系统不仅返回 `reply`，还返回 `trace`、`chartData`、`exportUrl` 和 `explain` 等字段。`reply` 面向普通用户，强调可读性；`trace` 面向可解释性和调试，说明识别意图、调用工具和处理步骤；`chartData` 面向前端图表展示；`exportUrl` 面向交易流水导出；`explain` 面向统计口径说明。通过这些结构化字段，前端可以把一次 AI 回复拆分为文本、图表、过程和操作入口，使系统更像一个完整的智能业务助手，而不是简单聊天窗口。

6.2 AI 语音输入功能实现

语音输入功能主要在前端 `AssistantPanel.vue` 中实现。组件初始化时调用 `initSpeechRecognition` 方法，优先读取浏览器提供的 `window.SpeechRecognition` 或 `window.webkitSpeechRecognition` 对象。如果当前浏览器支持该接口，系统创建 `speechRecognition` 实例，并设置 `lang` 为 `zh-CN`、`continuous` 为 `false`、`interimResults` 为 `true`、`maxAlternatives` 为 1。该配置说明系统面向中文短句输入，允许在识别过程中把临时结果及时显示到输入框中。

用户点击“语音输入”按钮后，`toggleVoiceInput` 方法会启动语音识别，并将 `voiceListening` 设置为 `true`。识别过程中，`onresult` 事件会读取 `event.results` 中的转写文本，并通过 `mergeSpeechText` 方法与原输入框内容合并，最终写入 `chatInput`。用户可以在识别后继续手动修改文本，也可以直接点击发送按钮。后续流程与普通文本输入完全一致，仍然由 `/assistant/chat/stream` 接口发送到后端，再由 `AssistantService` 进行意图解析、业务分发和结果封装。

该功能的设计重点是满足对语音识别交互的要求，并降低用户在移动或演示场景下的输入成本。由于语音识别只负责把语音转换为文本，不直接调用账户、流水、统计或转账服务，因此它不会改变原有业务安全边界。浏览器不支持语音识别、麦克风未授权或没有识别到语音内容时，前端会给出提示，用户仍可回到文本输入方式完成同一业务操作。

6.3 AI 查余额功能实现

AI 查余额对应 `balance` 意图。当用户输入“查一下我的余额”“我的账户信息”等表达时，`parseIntentByQwen` 会将 `intent` 解析为 `balance`。`AssistantService` 随后调用 `handleBalance` 方法，通过 `AccountServiceImpl.getAccountInfo` 查询当前登录用户的账户信息。

`handleBalance` 返回的内容包括户名、账号、账户类型、账户状态和余额。该功能实现虽较为直接，但它体现了系统设计中的关键原则：AI 只识别用户想查余额，余额数据本身必须来自数据库账户表，而不是来自模型生成。

余额查询功能虽然业务逻辑相对简单，但在本论文中具有基础性意义。该功能证明 AI 助手已经能够从自然语言入口进入真实后端服务，而不是只返回固定话术。同时还为后续流水查询、统计分析和转账功能提供了账户上下文基础。若余额查询无法保证数据准确，后续涉及金额分析和资金操作的功能就缺少可信前提。因此本文将其作为核心功能之一进行说明。

表 6.1 AI 查余额功能实现说明

输入示例	解析意图	调用方法	返回内容
查一下我的余额	<code>balance</code>	<code>handleBalance</code>	户名、账号、账户类型、状态、余额。
我的账户信息	<code>balance</code>	<code>handleBalance</code>	账户基本信息和余额。

6.4 AI 查流水功能实现

AI 查流水对应 `transactions` 意图。用户可以使用自然语言指定交易类型、时间范围和条数限制。例如“查最近 3 条支出记录”会被解析为 `txnType=支出`、`limit=3`、`timeRange=latest`；“查今天的流水”会被解析为 `timeRange=day`；“查前几天的支出记录”会经过 `Time.normalizeTimeByText` 纠偏为 `recent_days`。

`handleTransactions` 方法会根据解析结果构造 `TransactionQueryDTO`，并调用 `Time.fillTimeRange` 填充 `startTime` 和 `endTime`。随后系统调用 `TransactionServiceImpl.getTransactions` 进行查询，底层 SQL 由 `TransactionMapper.xml` 中的 `select-Transactions` 完成。查询结果由 `convertTransactionResultToChat` 格式化为自然语言列表，并对对方账号进行脱敏处理。

对于缺少月份的日期查询，系统设计了多轮补全机制。当 `needMonth` 为 `true` 且存在 `day` 但 `month` 为空时，系统将当前查询保存在 `pendingTxnQueryMap` 中，并提示用户补充月份。用户回复“1 月”后，下一轮请求会优先进入

handlePendingTransactionQuery，而不是被当作新的普通问题重新解析。该方法会读取待补全状态、解析月份、构造 TransactionQueryDTO，并继续调用流水查询服务完成查询。

流水查询比余额查询更能体现自然语言到结构化条件的转换过程。用户表达中可能包含交易方向、相对时间、绝对日期、条数限制等多种信息，后端需要把它们映射为 TransactionQueryDTO 中的字段。由于交易流水是账单统计和历史金额复用的基础数据，查询条件的准确性直接影响后续功能结果。本文在实现中采用模型解析和规则纠偏结合的方式：模型负责理解用户表达，Time 工具负责修正时间范围，Mapper 层负责执行确定 SQL 查询。

图6.2 以时序图形式展示了缺少月份时的两轮交互过程。第一轮中，用户提出“查 17 号收入记录”，系统识别为 transactions 意图并发现 month 为空，于是保存 PendingTransactionQuery 并追问月份；第二轮中，用户回复“1 月”，AssistantService 优先命中 pendingTxnQueryMap，补齐日期范围后调用 TransactionService 和 TransactionMapper 完成流水查询。

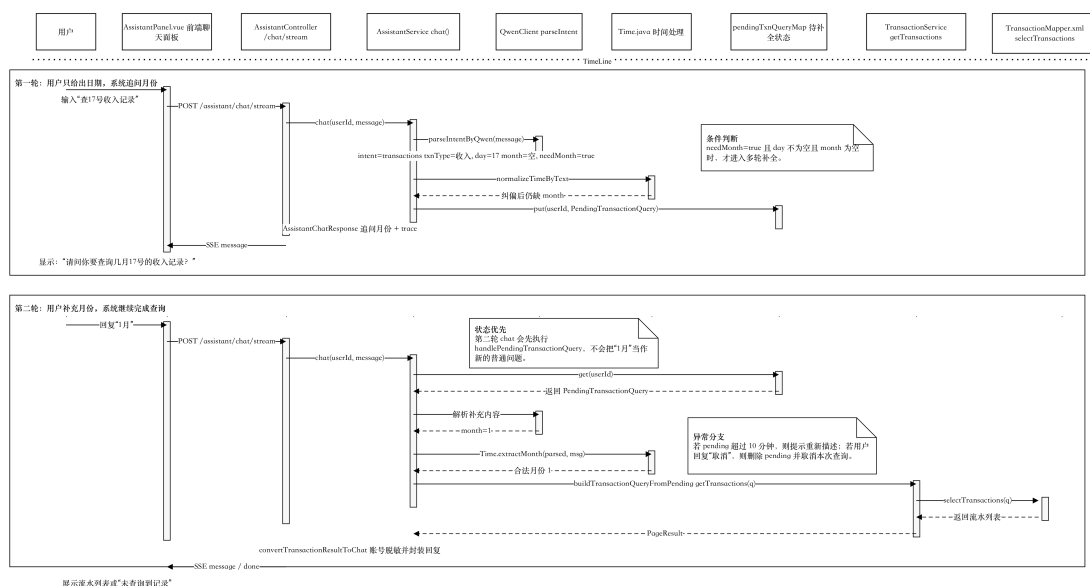


图 6.2 AI 查流水多轮补全时序图

表 6.2 AI 查流水测试输入与处理方式

用户输入	关键解析字段	系统行为
查最近 3 条支出记录	txnType= 支 出, limit=3, timeRange=latest	查询最近 3 条支出流水。
查今天的流水	timeRange=day	查询当天全部流水。
查前几天的支出记录	txnType= 支 出, timeRange=recent_days, rangeValue=3	查询近 3 天支出流水。
查 17 号收入记录	txnType= 收 入, day=17, need-Month=true	追问月份并进入待补全状态。

6.5 AI 账单统计与分析功能实现

AI 账单统计与分析对应 statistics 意图。系统支持 sum、count、max 和 trend 四类统计指标，分别用于统计总金额、流水笔数、最大一笔交易和趋势数据。用户输入“统计这个月总支出”时，系统会构造 txnType= 支出、metric=sum、timeRange=month 的 StatisticsQueryDTO；用户输入“看最近 7 天支出趋势”时，系统会构造 txnType= 支出、metric=trend、timeRange=recent_days、groupBy=day 的查询对象。

表 6.3 统计指标与实现方法对应关系

统计指标	含义	底层方法	前端展示
sum	总金额统计	sumAmount	数字结果或收支概览图。
count	笔数统计	countByStatistics	数字结果。
max	最大一笔统计	maxTransaction	数字结果。
trend	趋势统计	statisticsTrend	柱状图或折线图。

StatisticsServiceImpl 根据 metric 调用不同 Mapper 方法。sumAmount 用于总额统计，countByStatistics 用于笔数统计，maxTransaction 用于最大一笔交易，statisticsTrend 用于趋势聚合。当用户查询整体收支概览时，系统还会通过 sumIncomeAndExpense、statisticsByCounterparty 生成收支对比和按对方账号聚合的图表数据。因此，账单统计与分析本质上是对交易流水查询能力的进一步聚合和解释：流水查询返回明细记录，统计分析则在同一数据基础上完成汇总、趋势和概览。

统计分析文本的生成采用“事实构造 + 模型总结 + 规则兜底”的设计。

`AssistantService` 会将统计标题、统计类型、统计指标、时间范围、统计值、趋势明细、主要对方账号和异常信号等内容整理成 `facts`，然后请求通义千问生成自然语言分析总结。若模型输出为空、过于模板化或包含不支持的标签，消费类别等内容，系统会兜底返回规则化总结。与上一节多轮查流水不同，统计分析功能的重点不是多个参与对象之间的消息时序，而是自然语言条件如何被转换为统计 `DTO`，并经过确定性 `SQL` 聚合、事实整理和模型表达生成完整分析结果。

图6.3 展示了 AI 账单统计与分析的整体处理流程。系统先解析 `statistics` 意图并构造 `StatisticsQueryDTO`，再由 `StatisticsServiceImpl` 和 `TransactionMapper.xml` 完成总额、笔数、最大值或趋势等确定性统计。统计结果被封装为 `StatisticsResultVO` 后，`AssistantService` 进一步整理 `facts` 并调用大模型生成分析总结；当模型输出不符合约束时，系统使用规则化总结兜底，最后将 `reply`、`chartData`、`explain` 和 `exportUrl` 等字段返回前端展示。

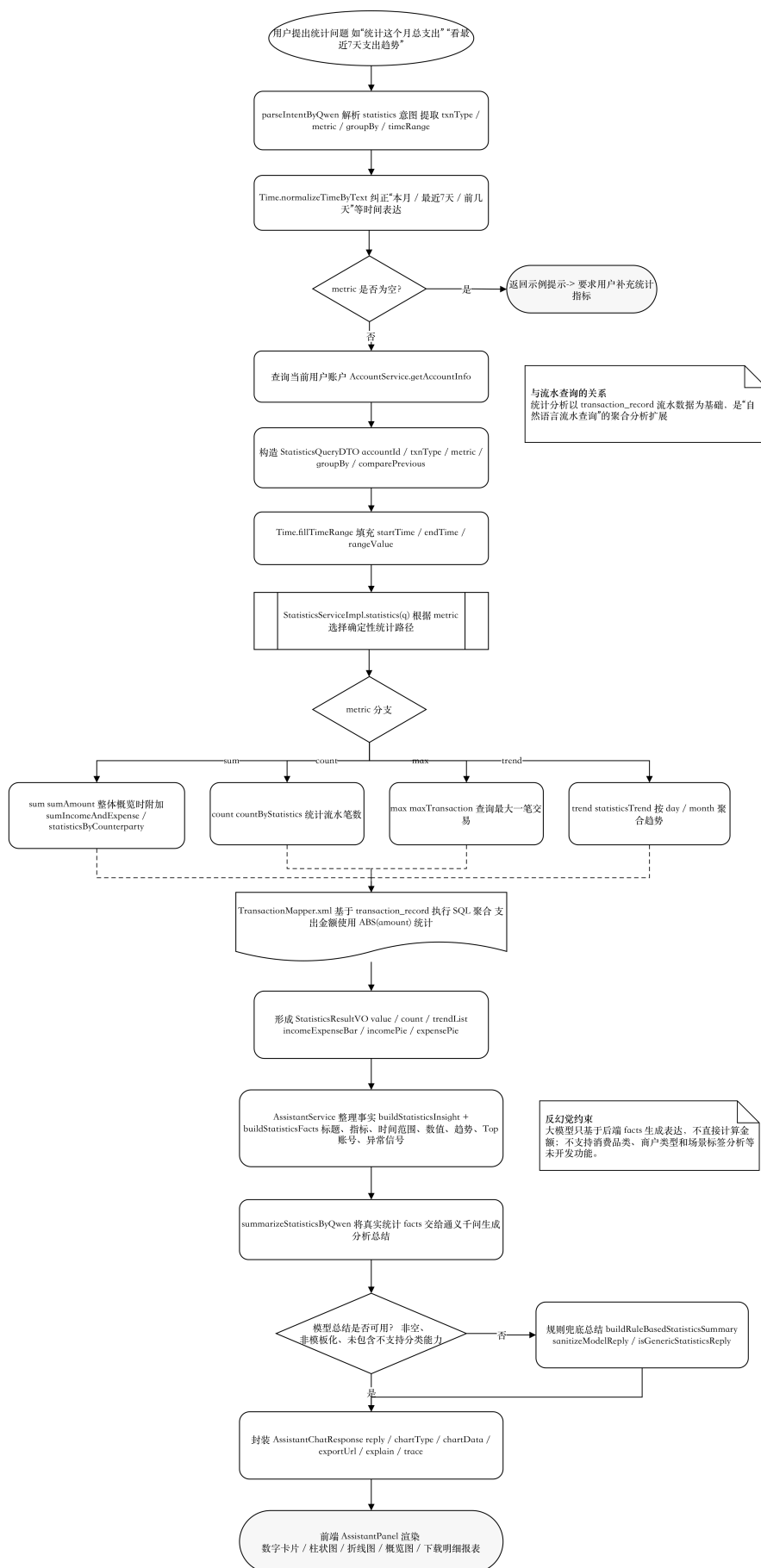


图 6.3 AI 账单统计与分析流程

前端根据 `AssistantChatResponse` 中的 `chartType` 和 `chartData` 渲染图表。`chartType` 为 `number` 时显示数字卡片，为 `bar` 或 `line` 时显示趋势图，为 `overview` 时显示收支对比柱状图和收入、支出按对方账号聚合的饼图。系统还会生成 `exportUrl`，用户可以下载当前统计口径下的交易流水 Excel 文件。

6.6 AI 周报/月报功能实现

AI 周报/月报由 `/assistant/brief` 接口提供，主要用于用户打开 AI 面板时自动展示财务概览。该功能不依赖用户主动提问，而是根据 `period` 参数生成 `week` 或 `month` 简报。

`getBrief` 方法会先查询当前用户账户，然后构造收入总额、支出总额和支出趋势三个统计查询。系统计算总收入、总支出和净流入，并进一步生成与上周或上月的对比数据。对于异常提醒，系统使用近 4 周平均支出和近 30 天支出金额 P90 阈值作为参考，检测当前周期支出是否明显偏高或是否存在超大单笔支出。

前端 `AssistantPanel` 在 `mounted` 阶段调用 `fetchAssistantBriefApi` 加载简报，并在面板顶部展示 `headline`、`highlights`、`suggestions`、`anomalies`、`comparison`、`quickActions` 和 `chartData`。用户可以在周报和月报之间切换，也可以点击快捷追问按钮继续向 AI 发起查询。

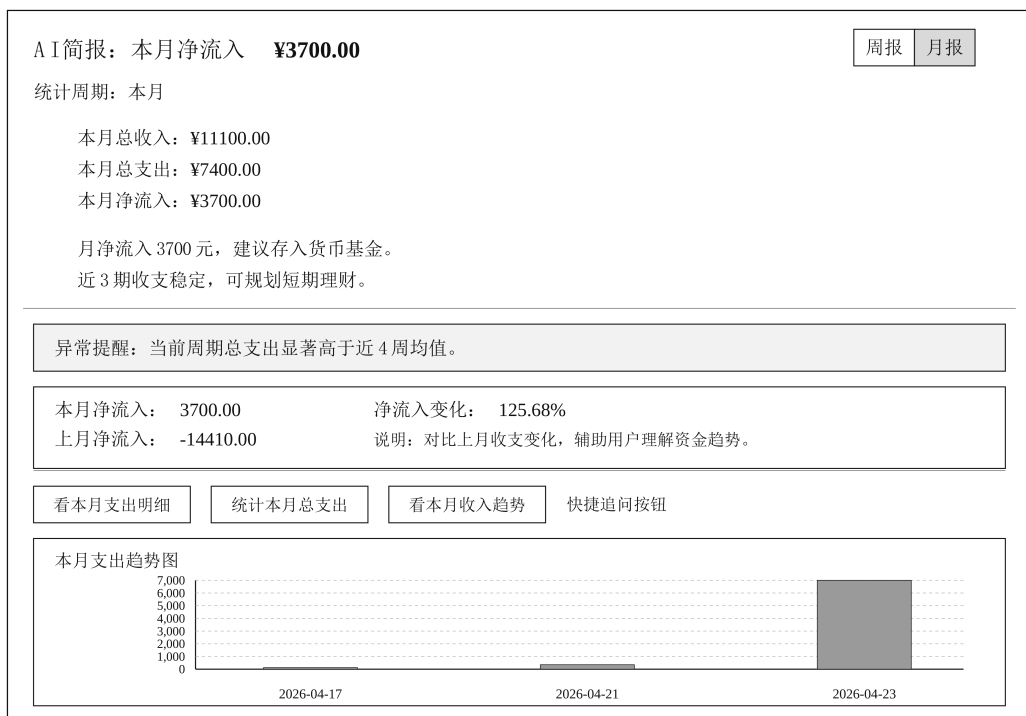


图 6.4 AI 月报卡片界面图

表 6.4 AI 周报/月报返回字段说明

简报字段	含义
headline	简报标题，例如 “AI 简报：本月净流入”。
highlights	本周或本月总收入、总支出、净流入。
comparison	当前周期与上一周期的收入、支出、净流入对比。
anomalies	异常支出提醒。
suggestions	AI 生成的简短建议。
quickActions	可直接点击发送的追问指令。
chartData	支出趋势图数据。

6.7 AI 智能转账功能实现

AI 智能转账对应 transfer 意图，是系统中安全要求最高的 AI 功能。用户可以直接提供银行卡号，也可以使用收款人昵称。例如 “向 6222000012345678 转 100 元” 会直接提取 toAccountNo 和 amount；“给弟弟转 300 元，备注午餐费” 会提取 payeeAlias= 弟弟、amount=300、description= 午餐费。

handleTransfer 方法首先解析收款账号、收款人昵称、金额、备注和 repeat-Mode。如果用户没有提供银行卡号但提供了昵称，系统会通过 PayeeContactMapper.selectByUserIdAndAlias 查询当前用户的收款人列表。若未找到昵称，系统提示用户先新增收款人或直接提供银行卡号；若匹配多个收款人，系统提示用户使用更具体昵称。

转账金额解析完成后，系统并不会立即执行扣款，而是调用 accountService.validateTransfer 或 payeeService.validateTransfer 进行校验。校验内容包括参数完整性、金额是否大于 0、付款账户是否存在、收款账户是否存在、账户是否冻结、是否给自己转账以及余额是否充足。校验通过后，系统将转账信息写入 PendingTransfer，并向用户返回脱敏后的确认信息。用户回复 “确认” 或点击 “确认” 按钮后，handlePendingTransfer 才会调用真实转账服务执行扣款、收款和流水写入；用户回复 “取消” 或点击 “取消” 按钮则删除待确认状态。

AI 智能转账涉及两轮用户交互和资金操作确认，如图6.5 所示，第一轮中用户通过自然语言发起转账，AssistantService 调用 QwenClient 解析 transfer 意图，并根据银行卡号或收款人昵称完成收款人匹配；随后系统调用转账校验服务检查账户状态、余额和自转账等风险条件。校验通过后，系统只生成确认信息并保存 PendingTransfer，不执行资金变动。第二轮中，用户回复 “确认” 或点击 “确认” 按钮时系统才调用真实转账服务更新账户余额并写入交易流水；用户回复 “取消”

或点击“取消”按钮或待确认状态超时则清理 pendingMap，资金操作不会发生。

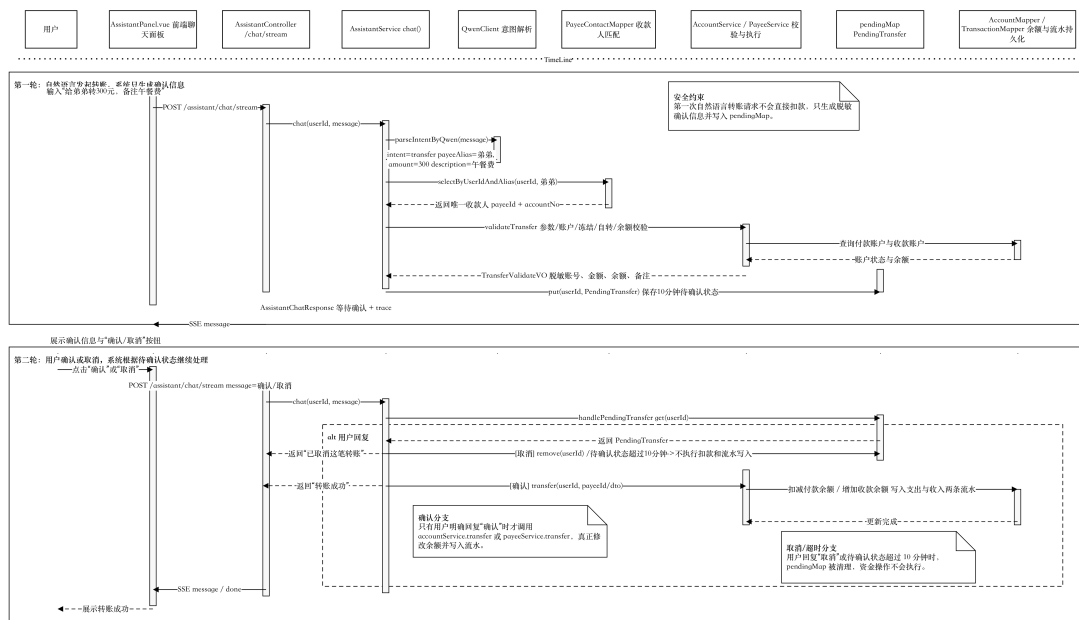


图 6.5 AI 智能转账二次确认时序图

系统同时实现了历史金额复用能力。当用户输入给弟弟转晚餐费并说明和前天一样时，模型会将 `repeatMode` 解析为 `time_same`，后端根据 `Time` 工具确定参考时间范围，并通过 `selectLatestExpenseByCounterpartyAndDescription` 查询该时间范围内同收款人且备注匹配的历史支出流水。若找到记录，系统复用历史金额和备注生成确认信息；若未找到，则提示用户补充转账金额。该能力只解决用户未明确给出金额时的金额补全问题，并不会绕过转账校验和二次确认。

图6.6 展示了历史金额复用转账的处理流程。系统首先判断当前 `transfer` 请求是否属于 `amount` 为空且包含时间参考的复用场景，然后解析收款人、备注关键词和参考时间范围。查询历史流水时，系统优先查找同收款人、同备注关键词的支出记录；若未找到，再回退查询同收款人的最近支出流水。只有找到真实历史流水时，系统才复用其金额和备注，否则要求用户补充金额。

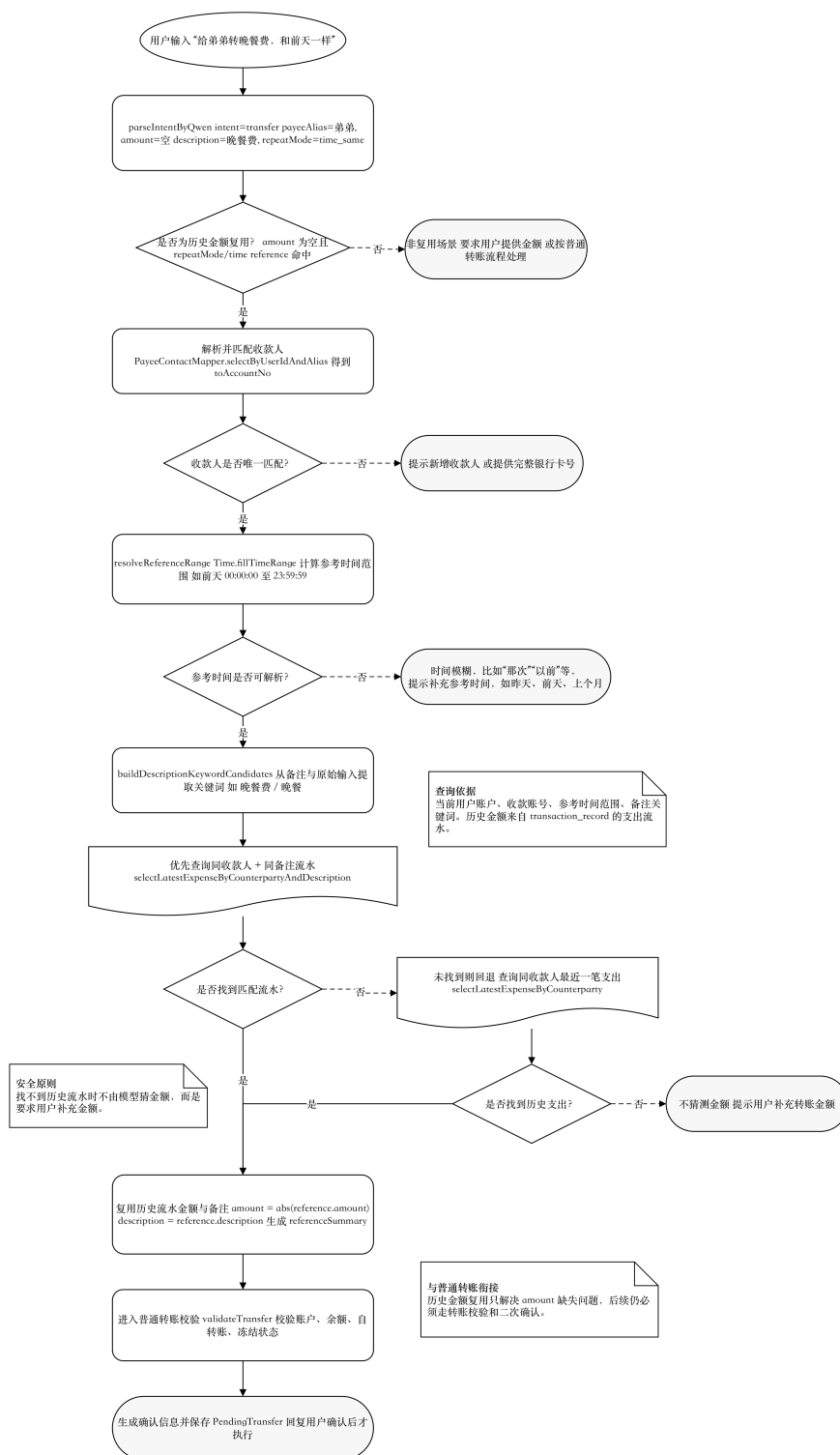


图 6.6 历史金额复用转账流程图

表 6.5 AI 智能转账安全控制设计

安全控制点	实现方式
收款人确认	按银行卡号或收款人昵称解析，昵称不存在或重复时提示用户补充。
账户校验	校验付款账户、收款账户是否存在及状态是否正常。
余额校验	转账前检查付款账户余额是否充足。
防止自转账	付款账号与收款账号相同时抛出业务异常。
二次确认	AI 首次回复只生成确认信息，用户回复确认后才执行。
账号脱敏	确认信息中付款账号和收款账号均以脱敏形式展示。

第七章 系统测试与交互优化分析

7.1 测试环境与测试方法

系统测试以本地开发环境为基础，后端服务运行在 8081 端口，前端通过 Vite 开发服务器代理 /api 请求到后端。数据库使用 MySQL，测试数据来自 sql 目录中的 user、account、payee_contact 和 transaction_record 脚本，数据库中数据信息均不涉及真实信息。测试方法以功能测试和交互流程测试为主，重点验证 AI 助手是否能够正确识别用户输入、调用对应业务服务、返回符合预期的文本、图表和确认流程。

本项目测试结果来自网上及校外专家导师提供的虚拟测试数据，交互优化分析重点从系统设计和功能可用性角度展开，不虚构用户满意度统计。

测试范围围绕本文定义的 AI 核心功能展开，不将登录、注册、普通页面跳转等基础管理能力作为主要评价对象。测试时首先保证后端服务、数据库和前端代理能够正常启动，然后在 AI 面板中输入典型自然语言指令，观察后端控制台、浏览器界面和接口返回结果是否一致。对于语音输入功能，重点检查识别结果是否能够回填到聊天输入框并继续发送；对于查询类功能，重点检查返回金额、记录条数、交易方向和时间范围是否与数据库样本相符；对于统计类功能，重点检查统计口径、聚合结果、图表数据和导出链接是否完整；对于转账类功能，重点检查系统是否先生成确认信息，是否在用户明确确认前避免直接修改账户余额。

为保证测试结论与源代码一致，本文没有设计超出项目能力范围的测试项。例如，当前系统并未实现商户品类识别和消费场景标签，因此测试只验证系统能否正确拒绝或说明该类请求，而不把这类功能写成已经完成的能力。对于大模型输出具有一定随机性的情况，测试判断以结构化字段和业务服务调用结果为主，而不是要求每次自然语言回复完全相同。这样可以避免把模型措辞差异误判为功能失败，也能更准确地反映系统的真实可用性。

7.2 意图识别准确性测试

为了验证本文所选择的大语言模型意图解析方案是否能够满足任务书中“选择适用的技术和算法”的要求，本文对 AssistantService 中的 parseIntentByQwen 方法进行了单独测试。测试时不执行余额查询、流水查询、统计计算和转账等后续业务分支，只检查模型返回的 intent 字段是否与人工标注意图一致。这样可以把

“自然语言理解是否正确”和“业务服务是否正确执行”区分开来。

测试样本覆盖 **balance**、**transactions**、**statistics**、**transfer** 和 **other** 五类意图，每类选取 10 条中文自然语言表达，共 50 条。样本既包含直接表达，如“查一下我的余额”“统计这个月总支出”，也包含口语化表达，如“我现在账户里还有多少钱”“最近几天的交易明细给我看看”“给弟弟转晚餐费，和昨天一样”。**other** 类样本包含系统不支持或非银行业务范围内的问题，如“按消费场景标签统计一下”“给我推荐一个理财产品”“今天西安天气如何”等。

表 7.1 意图识别准确性测试结果

意图类型	测试语句数	识别正确数	准确率	典型测试语句
查余额	10	10	100.0%	查一下我的余额
查流水	10	10	100.0%	查最近 5 条支出记录
账单统计	10	10	100.0%	最近 7 天我花了多少钱
智能转账	10	10	100.0%	给弟弟转 300 元午餐费
其它	10	10	100.0%	把消费按品类分类一下
合计	50	50	100.0%	典型样本集合

从表 7.1 可以看出，在本次 50 条典型测试样本中，系统能够正确区分余额查询、流水查询、统计分析、转账操作和不支持能力五类意图。测试结果说明，在本文设定的银行客服功能边界内，使用大语言模型进行意图识别能够满足原型系统的基本需求。

需要说明的是，该结果并不表示系统能够对任意开放式自然语言问题都保持 100% 识别准确率。本次测试样本主要围绕系统已实现功能和答辩演示高频问法构造，目的是验证项目源码中已设计的意图解析提示词、JSON 输出约束和不支持能力拦截是否有效。对于更大规模真实用户语料，仍需要继续扩大样本数量，并引入准确率、召回率、混淆矩阵和人工复核等更完整的评估方法。

7.3 功能测试

表 7.2 AI 核心功能测试用例表

编号	测试输入或操作	预期结果	结论
T1	输入“查一下我的余额”	识别为 balance ，返回账户信息和余额。	通过
T2	输入“查最近 5 条支出记录”	识别为 transactions ，返回支出流水列表。	通过
T3	输入“查 17 号收入记录” 后回复“1 月”	系统先追问月份，再查询指定日期收入流水。	通过
T4	输入“统计这个月总支出”	识别为 statistics ，返回支出总额、统计口径和导出链接。	通过
T5	输入“看最近 7 天支出趋势”	返回趋势图数据，前端可渲染柱状图或折线图。	通过
T6	打开 AI 面板	自动加载周报或月报卡片。	通过
T7	输入“给弟弟转 300 元，备注午餐费”	按昵称匹配收款人并返回转账确认信息。	通过
T8	在待确认转账状态下输入“取消”	取消转账并清除待确认状态。	通过
T9	输入“给弟弟转 200 元晚餐费，和昨天一样”	识别为 transfer ，按昵称匹配收款人，解析金额和备注，并生成待确认转账信息。	通过
T10	提出消费分类或商户类型分析请求	系统说明当前不支持该类能力。	通过
T11	点击语音输入按钮并说出“查一下我的余额”	浏览器完成语音转写，输入框出现识别文本，发送后进入余额查询流程。	通过

从测试结果看，系统能够覆盖论文设计的语音输入和五项核心 AI 业务功能。对于语音输入任务，系统能够把用户语音转换为可编辑文本；对于常规查询类任务，系统能够通过意图解析进入对应业务流程；对于缺失信息的查询，系统能够通过多轮对话补全；对于转账类任务，系统能够在执行前给出确认信息并等待用户明确操作。

语音输入测试主要验证前端输入入口。用户点击语音输入按钮后，浏览器请求麦克风权限并启动 **SpeechRecognition**。识别到中文语音后，前端将转写文本回填到 **chatInput** 中，用户可继续修改或直接发送。该测试说明语音识别已经接入智能客服交互入口，但其后续业务处理仍然遵循文本请求链路，不会绕过后端意图解析和安全校验。

余额查询测试主要验证账户数据读取链路。用户输入余额查询请求后，后端将其解析为 **balance** 意图，并调用账户相关服务读取当前用户账户信息。测试中需要关注两点：一是回复内容不能脱离数据库真实余额自行生成；二是当账户不存在或数据异常时，系统应返回明确提示，而不是给出含糊的正常结果。该测试说明余额查询功能已经具备作为智能客服基础能力的条件。

流水查询测试主要验证自然语言条件到查询参数的映射。系统需要从“最近 5 条支出记录”“17 号收入记录”等表达中识别交易方向、数量限制和日期信息。对于日期不完整的表达，系统不会直接猜测月份，而是进入多轮补全流程，要求用户进一步说明。该处理方式比直接推断更稳妥，能够降低查询范围错误导致的用户误解。测试结果表明，系统可以在常见流水查询表达下生成可执行条件，并正确返回流水列表。

账单统计与分析测试主要验证数据库聚合和图表数据生成。系统接收到“统计这个月总支出”“最近 7 天支出趋势”等请求后，会构造 **StatisticsQueryDTO**，并由后端统计服务完成聚合计算。大模型不直接计算金额，而是在后端已经形成事实数据的基础上生成说明性文本。测试中，如果统计结果为空，系统也需要说明当前范围内没有相关记录。该设计符合银行业务对数据真实可靠的要求。

周报/月报测试主要验证系统主动概览能力。用户打开 AI 面板后，前端会请求简报数据，并将收入、支出、净流入、趋势和快捷追问组织为卡片。该能力的价值在于用户不必主动输入复杂问题，也能快速了解近期账户变化。测试时重点检查简报统计周期是否正确、卡片数据是否完整、快捷追问是否能够继续进入 AI 聊天流程。

智能转账测试是安全性要求最高的部分。系统需要识别收款人、金额和备注，并根据常用收款人信息匹配真实收款账号。对于 T7 这类已经给出明确金额的请求，系统以用户显式输入的金额为准，完成收款人匹配、金额解析、备理解析和转账前校验后，只生成待确认信息，不直接执行扣款。对于 T8 场景，系统能够识别取消意图，清除 **pendingMap** 中保存的待确认转账状态，保证用户取消后不会继续保留旧的资金操作请求。对于 T9 这类未提供明确金额但带有时间参考的请求，系统才会进入历史金额复用逻辑：先根据收款人昵称确定收款账号，再结合备注关键词和参考时间范围查询历史支出流水；若查到匹配记录，则复用历史金额生成确认信息，若未查到记录，则要求用户补充金额。测试结果表明，智能转账功能在自然语言解析、常用收款人匹配、历史金额复用、确认执行和取消回退等分支上均能保持“确认前不执行”的安全原则。

7.4 交互优化设计分析

第一，系统采用流式响应降低等待感。前端调用 `/assistant/chat/stream` 后，后端先返回 `trace` 事件，提示“请求已接收”和“正在识别业务意图”，最终再返回 `message` 事件。相比普通请求一次性返回结果，流式处理能让用户知道系统正在处理请求，减少等待过程中的不确定性。

第二，系统展示 AI 处理过程提升可解释性。`AssistantChatResponse` 中的 `trace` 字段包含 `intent`、`tool`、`status`、`confidence` 和 `steps`。前端将这些信息展示在聊天气泡中，使用户和答辩评阅者能够看到系统识别了什么意图、调用了什么能力以及处理状态。

第三，系统通过图表增强统计结果表达。对于趋势类统计，用户不仅能看到文字总结，还能看到柱状图或折线图。对于收支概览，系统能够展示收支对比和按对方账号聚合的饼图。图表展示降低了用户理解复杂流水数据的成本。

第四，系统提供语音输入减少文字输入成本。用户点击语音输入按钮后可以直接说出查询或转账需求，前端将语音识别结果填入输入框。该设计对演示场景较为直观，也使语音识别要求落到实际交互功能中。

第五，系统提供快捷操作减少输入成本。简报卡片提供快捷追问按钮，用户可以一键继续查询；转账确认场景中显示“确认”和“取消”按钮，减少用户手动输入。

第六，系统对不支持能力进行边界控制。当前项目并未实现消费品类、商户类型和场景标签分析，因此后端在意图解析后会对这些需求进行拦截，避免大模型给出超出系统能力的结论。这种设计有利于提高银行场景下的可信度。



图 7.1 AI 处理过程展示界面图

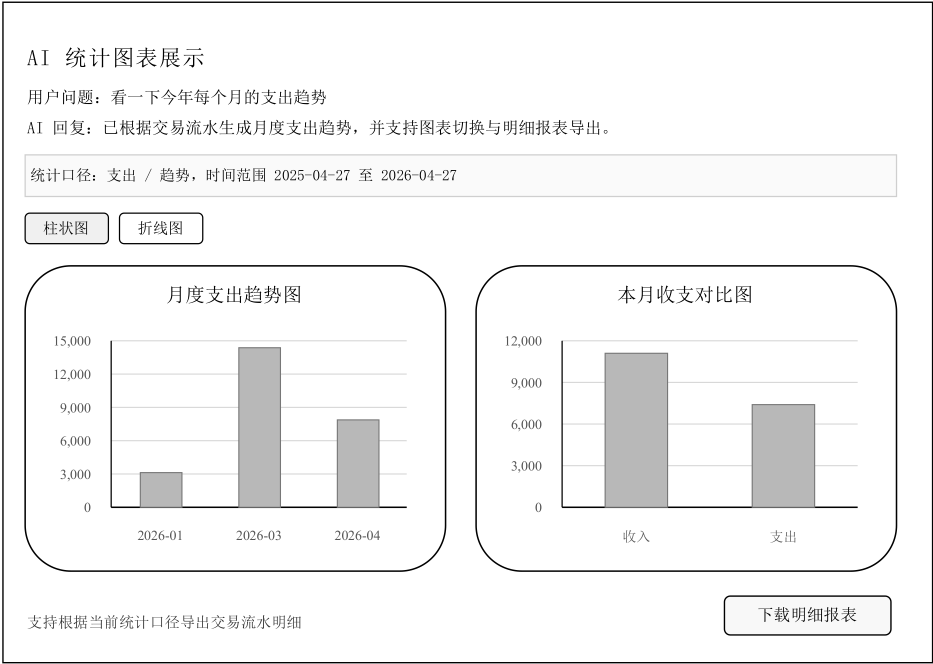


图 7.2 AI 统计图表展示界面图

7.5 测试结论

综合功能测试和交互分析，BankCopilot 智能客服机器人能够较好地完成文本输入和语音转文字输入形式的银行业务交互。系统在查询类任务中能够正确调用账户和流水服务，在统计类任务中能够基于数据库聚合结果生成图表和分析，在转账类任务中能够通过二次确认保证资金操作安全。虽然目前系统核心功能较少，但其链路完整，涉及场景典型，整体设计体现了智能客服机器人在银行场景中的可行性和实用价值。

从测试结论看，本文系统已经完成了从自然语言输入到业务服务调用、再到结构化结果展示的闭环。其主要优势体现在三个方面：第一，系统没有把银行业务结果完全交给大模型生成，而是以真实数据库和后端服务为依据；第二，系统对查询、统计和转账采用不同交互策略，能够根据业务风险调整流程复杂度；第三，系统在展示层面保留 AI 处理过程和统计口径说明，便于用户理解系统为何给出某一结果。

同时，测试也反映出当前系统仍属于原型验证阶段。测试数据规模较小，尚不能覆盖真实银行系统中复杂的账户状态、异常交易、并发转账和风控规则。如果将系统扩展到更真实的业务环境，还需要补充自动化测试、压力测试、安全测试和异常恢复测试。本文在结论中将这些问题作为不足进行说明，避免夸大系统当前能力。

第八章 结论与展望

8.1 主要结论

本文围绕“智能客服机器人的设计与交互优化”这一课题,完成了 BankCopilot 银行智能客服机器人原型的设计与实现。作为本科计算机专业毕业设计,本系统的定位不是构建完整商业银行系统,而是在一个规模可控的银行管理系统中,验证 AI 能力如何与传统业务管理系统结合。系统整体功能不追求大而全,而是围绕语音输入、余额查询、流水查询、账单统计、周报/月报和智能转账等典型场景,形成一个小而完整、边界清晰、能够演示的 AI 业务闭环。

从功能设计看,各项 AI 能力承担的业务角色并不重复。AI 语音输入主要体现自然语言输入方式的扩展,用户可以通过语音说出需求,再由前端转写为文本进入同一聊天链路。AI 查余额主要体现自然语言到数据库单对象查询的转换,用户用口语化方式提出问题,后端仍然从账户表读取真实余额和账户状态。AI 查流水体现自然语言条件到列表查询条件的转换,系统需要识别交易方向、条数限制和时间范围。AI 账单统计与分析体现数据库聚合能力,系统将流水数据进一步转换为总额、笔数、趋势和对比结果。AI 周报/月报是在统计结果基础上的概览展示,强调主动汇总和快捷追问。AI 智能转账则对应传统管理系统中最典型的流程型操作,需要按照收款人匹配、金额解析、业务校验、待确认、确认执行或取消的步骤逐步完成。

从系统实现看, AI 与传统管理系统结合时,关键不在于让模型直接完成所有业务,而在于明确模型和后端系统的职责边界。大语言模型适合完成自然语言理解、意图识别和结果表达,但账户数据、流水数据、统计结果和转账执行必须由 Spring Boot 后端和 MySQL 数据库提供确定性支持。对于银行业务这类需要准确性和可追溯性的场景,时间范围、统计指标、交易方向、金额、收款人和确认状态等条件都必须被明确约束,不能由模型自由推断。

从论文创新性看, BankCopilot 的创新不在于提出新的大模型算法,而在于将大语言模型能力嵌入到一个传统银行管理系统的具体业务流程中,形成了较完整的工程化应用方案。第一,系统将 AI 定位为自然语言入口和交互增强层,用户可以通过文本或语音转文字方式触发查询、统计和转账等传统业务操作,但最终业务结果仍由后端服务和数据库生成,避免了模型直接编造业务数据。第二,系统选取的功能具有层次差异:语音输入对应输入方式扩展,查余额对应基础数据查询,查流水对应条件查询,账单统计对应聚合计算,周报/月报对应结果组织和主

动展示，智能转账对应有状态、有约束、有确认的流程型操作，能够比较完整地体现 AI 与管理系统结合时的不同业务形态。第三，系统在实现中引入了时间范围解析、统计口径解释、待确认状态、历史金额复用、账号脱敏和能力边界控制等机制，使自然语言交互不只是简单问答，而是能够落到可控、可追踪的业务执行流程中。对于本科毕业设计而言，这种创新更偏向系统设计和工程集成创新，体现了前后端分离、数据库设计、接口设计、自然语言解析、业务服务调用、图表展示和测试验证等计算机专业综合能力。

8.2 实践价值

BankCopilot 的实践价值主要体现在智能客服能力与银行业务流程的结合。传统页面更适合结构化操作，但当用户目标明确而路径复杂时，自然语言入口可以减少页面查找和条件设置成本。例如，用户只需要输入“查最近 5 条支出记录”，系统即可完成意图识别、条件转换、流水查询和结果展示。对于账单统计，系统能够把分散流水转化为总额、趋势和概览，使用户更容易理解账户变化。

在安全性方面，本文系统没有因为引入 AI 而弱化业务控制。尤其在转账功能中，系统把自然语言解析结果转化为待确认信息，并在确认前保持状态而不执行资金变动。这种方式既保留了智能交互的便利性，又符合资金类业务需要用户明确授权的原则。对于毕业设计展示而言，该功能能够较直观地体现“智能”并不等同于自动越权执行，而是要在用户授权、业务规则和交互体验之间取得平衡。

在可解释性方面，系统通过 trace 信息展示意图、工具、状态、置信度和处理步骤。该设计使 AI 处理过程从黑盒回复变为可观察链路，便于用户理解系统处理逻辑，也便于教师在评阅时确认系统不是简单静态问答。对于账单分析和统计图表，系统同时给出统计口径说明，避免用户把不同时间范围、不同交易方向或不同聚合方式下的结果混淆。

8.3 不足与展望

受本人开发经验、项目周期和测试条件限制，当前系统仍然存在一些不足。首先，多轮对话状态主要保存在后端内存中，服务重启后会丢失，这种方式适合毕业设计原型展示，但距离长期稳定运行还有差距。其次，系统依赖外部大模型服务，当网络异常或模型接口不可用时，意图解析和分析总结能力会受到影响，虽然已有部分规则兜底，但还不够完善。再次，当前交易流水数据字段较简单，系统能够完成余额查询、流水查询、统计分析和转账确认，但还不能深入分析消

费品类、商户类型和具体消费场景。

如果后续有机会继续完善该系统，我认为可以从几个较实际的方向改进。第一，可以将待确认转账、待补全查询等会话状态保存到 **Redis** 或数据库中，提高系统重启后的恢复能力。第二，可以继续补充异常情况处理和测试用例，例如模型返回格式异常、网络请求失败、余额不足、收款人不存在等场景，使系统在演示之外更加稳定。第三，可以在交易流水中增加更丰富的字段，并在此基础上扩展消费分类、预算提醒和更细粒度的账单分析功能。通过这些改进，**BankCopilot** 可以在现有毕业设计原型的基础上进一步完善，但就本次毕业设计而言，系统已经完成了 **AI** 与传统银行管理系统结合的主要设计目标。

第九章 结束语

本文围绕个人银行业务中的智能客服交互问题，完成了从需求分析、总体设计、核心功能实现到测试分析的完整论述。论文写作过程中始终以本人独自完成的项目源码为依据，只对系统中已经实现并能够演示的功能进行说明，没有将未完成的功能写入系统成果。系统的重点不是构建一个泛化聊天机器人，而是在一个具体的银行管理系统中，验证大语言模型与确定性业务服务结合的实现方式。

从实现结果看，BankCopilot 已经能够处理语音输入、余额查询、流水查询、账单统计、周报/月报和智能转账等典型场景。系统通过语音转文字和自然语言意图识别降低用户操作门槛，通过后端业务服务保证数据来源和资金操作的确定性，通过处理过程展示、统计口径说明和转账二次确认增强交互可解释性与可控性。虽然该系统仍然存在测试数据规模有限、风控规则较简单、会话状态未持久化等不足，但作为本科毕业设计原型，已经较完整地展示了 AI 能力嵌入传统管理系统后的基本流程和实现效果。

通过本课题的完成，本人对前后端分离开发、数据库建模、接口设计、自然语言意图识别、流式交互和 LaTeX 论文写作有了更系统的认识。整个开发和论文整理过程也让我认识到，AI 功能落地到具体业务系统时，不能只关注模型回复是否自然，更需要关注数据来源、业务约束、异常处理和用户确认等工程细节。本次毕业设计仍有不足，但基本达到了任务书对系统设计、功能实现和论文撰写的要求。

致谢

本论文和系统原型的完成离不开指导教师和企业专家导师在选题方向、系统设计和论文写作方面的指导。在项目实现过程中，相关课程学习和开源技术文档为系统架构设计、前后端开发和数据库实现提供了重要帮助。同时，感谢在毕业设计过程中给予建议和支持的指导教师、企业专家导师，辅导员和同学。由于本人能力和时间有限，论文和系统仍存在不足之处，恳请各位批阅老师批评指正。

参考文献

- [1] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems: Vol. 30. 2017.
- [2] BROWN T B, MANN B, RYDER N, et al. Language models are few-shot learners[C]//Advances in Neural Information Processing Systems: Vol. 33. 2020: 1877-1901.
- [3] ZHAO W X, ZHOU K, LI J, et al. A survey of large language models[A/OL]. arXiv (2023). <https://arxiv.org/abs/2303.18223>.
- [4] ALCAZAR J, BAIRD S, CRONENWETH E, et al. Core banking systems and options for modernization[R]. Kansas City: Federal Reserve Bank of Kansas City, 2024.
- [5] CRISANTO J C, LEUTERIO C B, PRENIO J, et al. Regulating AI in the financial sector: recent developments and main challenges: FSI Insights No. 63[R]. Basel: Bank for International Settlements, 2024.
- [6] Bank of America. A decade of AI innovation: BofA's virtual assistant erica surpasses 3 billion client interactions[M/OL]. Charlotte: Bank of America Newsroom, 2025. <https://newsroom.bankofamerica.com/content/newsroom/press-releases/2025/08/a-decade-of-ai-innovation--bofa-s-virtual-assistant-erica-surpas.html>.
- [7] JPMorgan Chase & Co. 2016 annual report[M/OL]. New York: JPMorgan Chase & Co., 2017. <https://www.jpmorganchase.com/content/dam/jpmc/jpmorgan-chase-and-co/investor-relations/documents/2016-annualreport.pdf>.
- [8] China Merchants Bank. 2024 annual report[M/OL]. Shenzhen: China Merchants Bank, 2025. https://pdf.dfcfw.com/pdf/H2_AN202503251647351562_1.pdf?1742926197000.pdf.
- [9] Bank for International Settlements. Managing explanations: how regulators can address AI explainability[R]. Basel: Bank for International Settlements, 2025.
- [10] National Institute of Standards and Technology. Artificial intelligence risk management framework: AI RMF 1.0: NIST AI 100-1[R]. Gaithersburg: National Institute of Standards and Technology, 2023.
- [11] International Committee on Credit Reporting. Credit scoring approaches guidelines[R]. Washington, DC: World Bank Group, 2020.
- [12] LEO M, SHARMA S, MADDULETY K. Machine learning in banking risk management: a literature review[J]. Risks, 2019, 7(1): 29.
- [13] Spring Team. Spring boot reference documentation[M]. Palo Alto: VMware, 2025.
- [14] Vue.js Team. Vue.js guide[M]. Online: Vuejs.org, 2025.
- [15] Apache ECharts Team. Apache ECharts documentation[M]. Wilmington: Apache Software Foundation, 2025.
- [16] MyBatis Team. MyBatis 3 user guide[M]. Online: MyBatis.org, 2025.

附录 A 主要接口与数据结构

A.1 AI 聊天请求结构

AI 聊天请求由 AssistantChatRequest 承载，核心字段为 message。用户自然语言输入会以 JSON 请求体形式发送至 /assistant/chat 或 /assistant/chat/stream 接口。

表 A.1 AssistantChatRequest 请求字段

字段	类型	说明
message	String	用户自然语言输入，例如“统计这个月总支出”。

A.2 AI 聊天响应结构

表 A.2 AssistantChatResponse 响应字段

字段	类型	说明
reply	String	AI 回复文本。
chartType	String	图表类型，如 number、bar、line、overview。
chartData	Object	图表数据或数字统计结果。
exportUrl	String	导出交易流水的链接。
trace	TraceInfo	AI 处理过程信息。
explain	Map	查询或统计口径说明。

A.3 统计查询结构

表 A.3 StatisticsQueryDTO 字段说明

字段	说明
accountId	账户 ID。
txnType	收入或支出。
metric	统计指标，支持 sum、count、max、trend。
groupBy	分组方式，支持 none、day、month。
startTime	统计开始时间。
endTime	统计结束时间。
rangeValue	最近 N 天或 N 年的数值。
comparePrevious	是否对比上一周期。